Swap and Memory Reclaim

Squeezing Out More RAM

Kairui Song - Tencent
ryncsn@gmail.com / kasong@tencent.com



Swap Subsystem

- Allows you to use more memory than you have.
- Use storage as if they were RAM
- Use resoruces that’s not directly addressable by the CPU as RAM (page fault)

- Memory compression, which is increasingly common nowadays.
- Swap in Linux has a long history and burden, with many updates and improvements over the past few years and continuing today.
- A failsafe when you run out of memory. Or, Secondary memory.

```
root@POTATO:~# memhog 64G >/dev/null && echo Success!
numactl: mmap: Cannot allocate memory
root@POTATO:~# memhog 62G >/dev/null && echo Success!
Killed                       memhog 62G > /dev/null
```

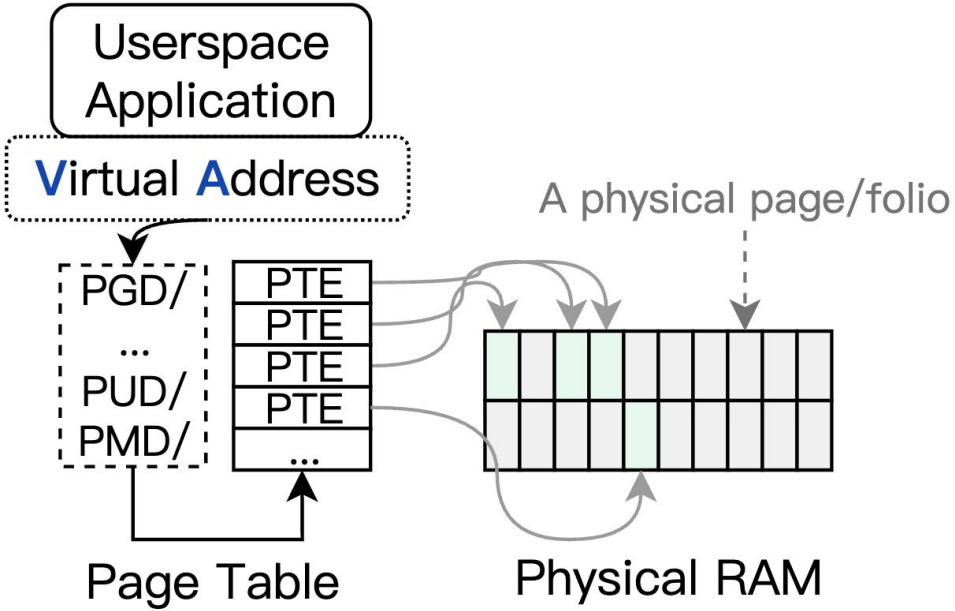
```
root@POTATO:~# swapon /dev/nvme1n1p4
root@POTATO:~# memhog 64G >/dev/null && echo Success!
Success!
```

```
root@POTATO:~# swapon /dev/zram0
root@POTATO:~# memhog 64G >/dev/null && echo Success!
Success!
```

How Swap Works

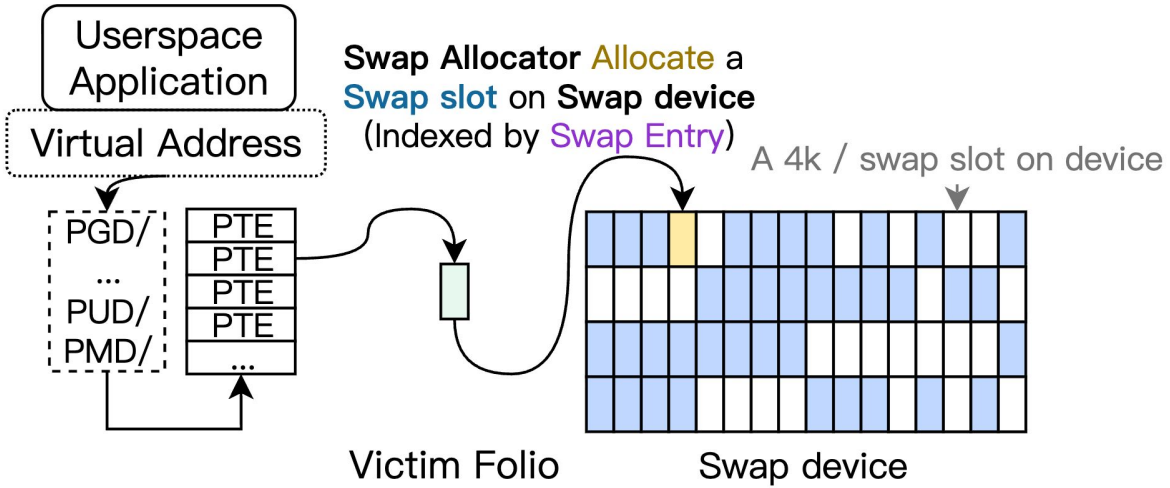
- When under memory pressure, we want more free **physical memory**.

- Userspace uses **virtual addresses**. The CPU walks the page table -> **PTE** points to the **physical memory** (Folio, Mapped).
- The kernel can **manipulate the physical memory** as long as userspace still sees the same virtual memory.
- Swap: **frees physical memory**, stores its content elsewhere, and brings it back when the application needs it again.



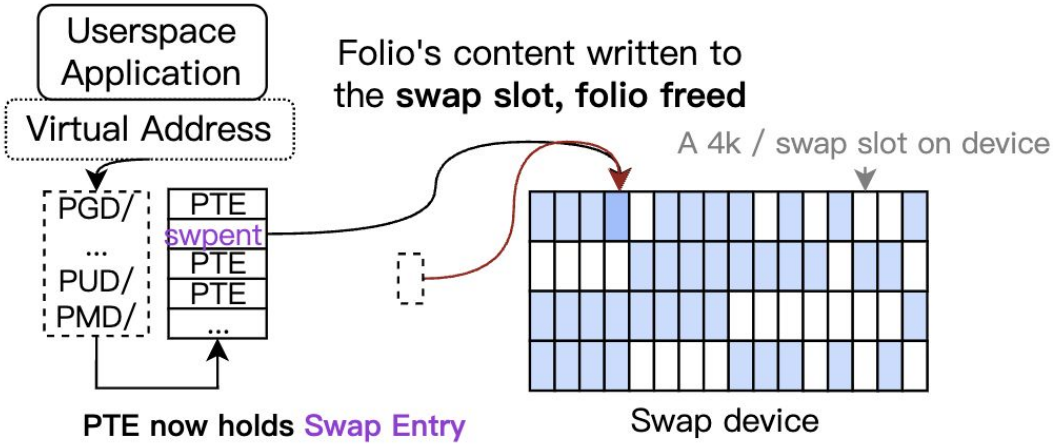
Swap Basics: Swap Out

- Assuming 4K folios, 64 bits, a swap device is mounted, details simplified.
- Linux Kernel selects a victim folio.
- **Swap allocator**: allocates a **slot** (4K) on the swap device.
- The slot is represented as a **swap entry.**
- Swap entry encoding: device id (type) and offset.



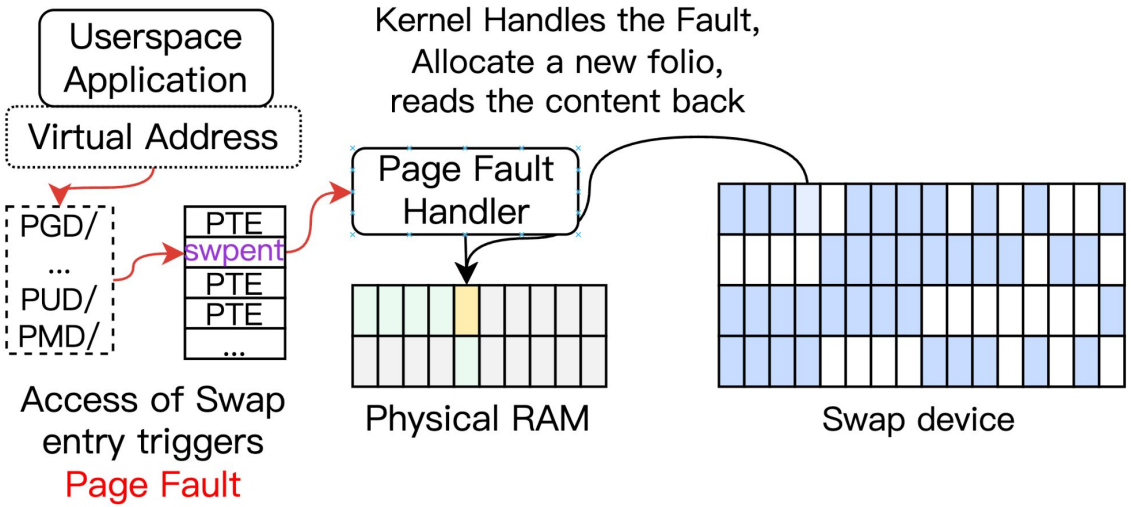
Swap Basics: Swap Out

- The folio is **unmapped** from the page table, the **swap entry** is stored in the page table instead (using arch specific encoding)
- The folio's content is **written back** to the swap slot.
- The folio is **freed**, memory pressure is released.
- **Swap out done**.



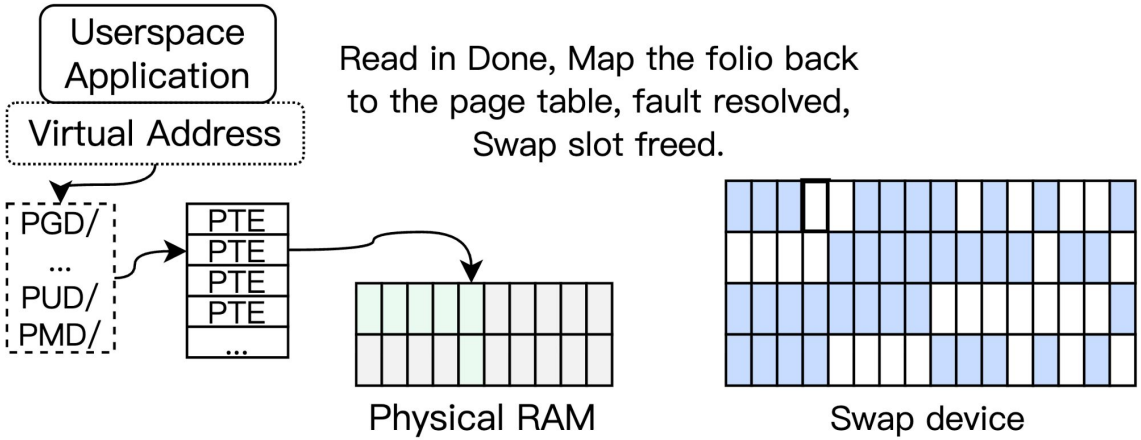
Swap Basics: Swap in

- Accessing the swap entry in the page table triggers a **page fault**
- Linux kernel's Page Fault handler gets the swap entry, allocates a new physical folio, read the content back.



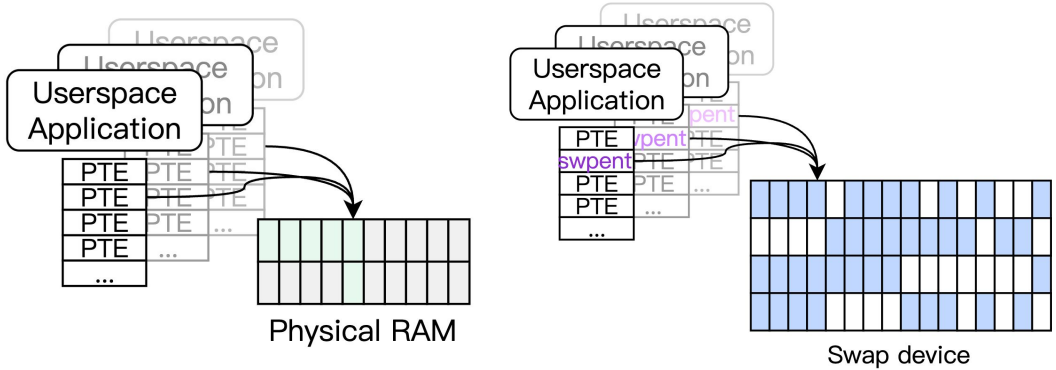
Swap Basics: Swap In

- The folio is mapped back to page table, virtual memory has the same content as before.
- Swap slot is freed.
- **Swap in done**.



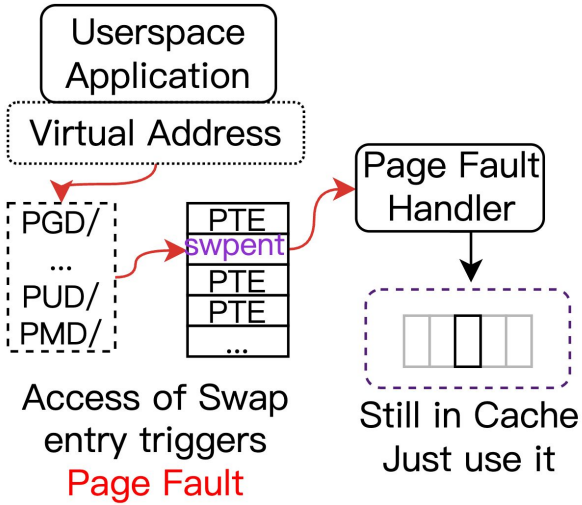
Reality is much more complex…

- Multiple PTEs can point to a single folio (e.g., via CoW or KSM)
- The same applies to swap slots.
- We need to keep a **reference count** for each swap slot so it is freed only after all users are gone.
- We need a cache to avoid redundant I/O and handle synchronization.



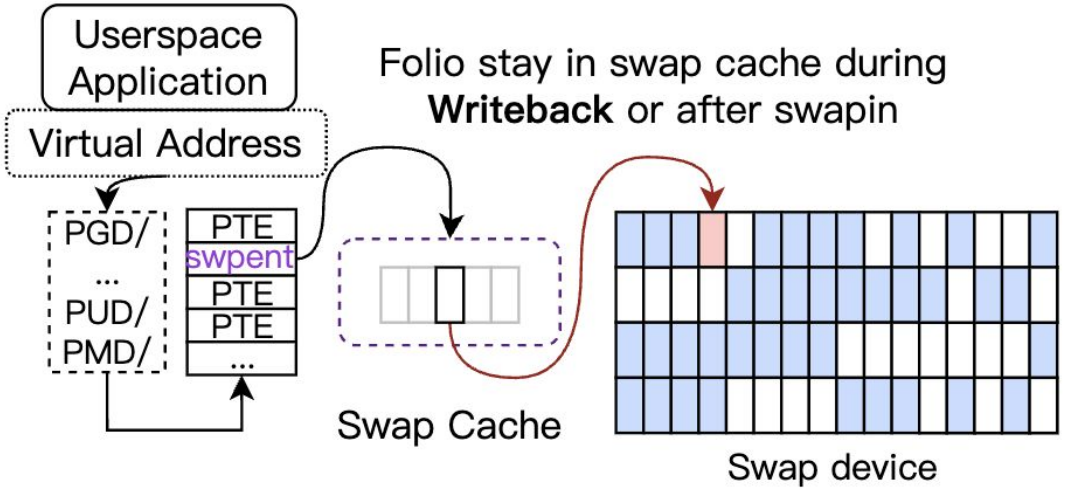
With Swap Cache

- During **swap in**, the new allocated folio stays in **swap cache**.
- The swap cache is **a map indexed by the swap entry**.
- If another **concurrent swap-in** needs the same swap entry, it will find the folio in the swap cache, and wait on folio lock.
- This **saves swap-in I/O** and greatly **simplify synchronizes** of concurrent swap-in.
- Helpful even for single mapped folio / swap slot, multiple thread can try swap in the same address.



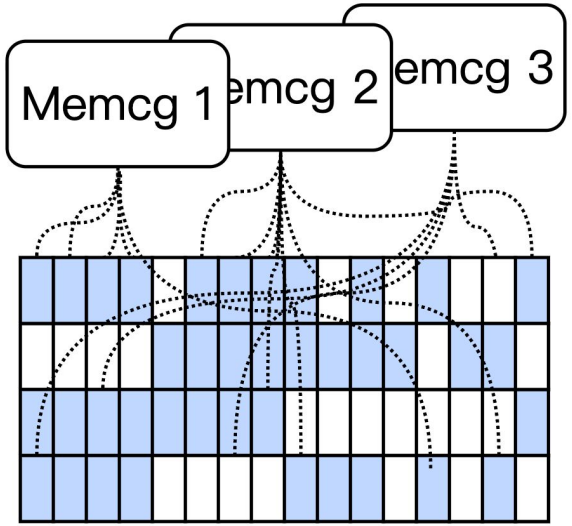
With Swap Cache

- During **swap out**, the folio also remains in the swap cache.
- If the folio is requested again, it enables fast recovery.
- Also, the **swap cache is lazily freed** after swap-in.
- So if a swap-out request hits a clean folio in the swap cache, swap-out I/O is saved.



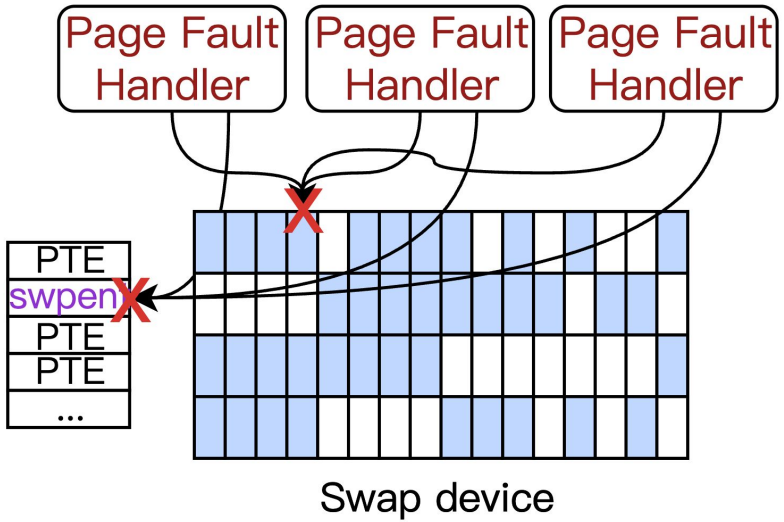
Reality is even more complex…

- Swap is a core component of memory management, so memory cgroups account for it too.
- Therefore, we need to track memcg for each slot.



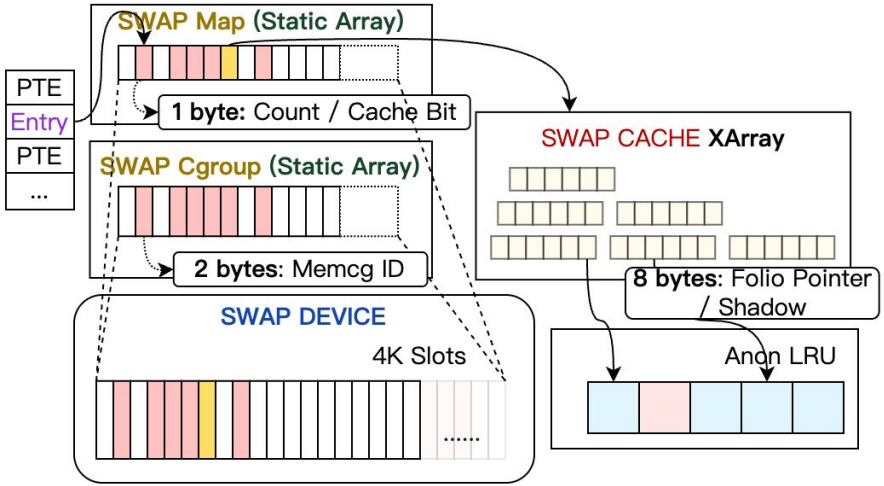
Reality is even more complex…

- Like every MM component, swap is always under high pressure and high concurrency.
- **Performance sensitive** and lots of contention or race.



Reality is even more complex…

- The data just kept piling up.
  - **count**: swap_map 1 byte - **static**
    - 6 bits usable, overflow & sync flag
  - **memcg id**: 2 bytes - **static**
  - **swap cache folio ptr / shadow**: 8 bytes - **xarray**
  - (overflow page, zeromap, etc.)
- **swap_map was the core**, allocation is just scanning it, scales poorly.
- Synchronization between multiple data sources became a burden, resulting in poor locking, lower performance, bugs.
- Static data, high idle memory usage.



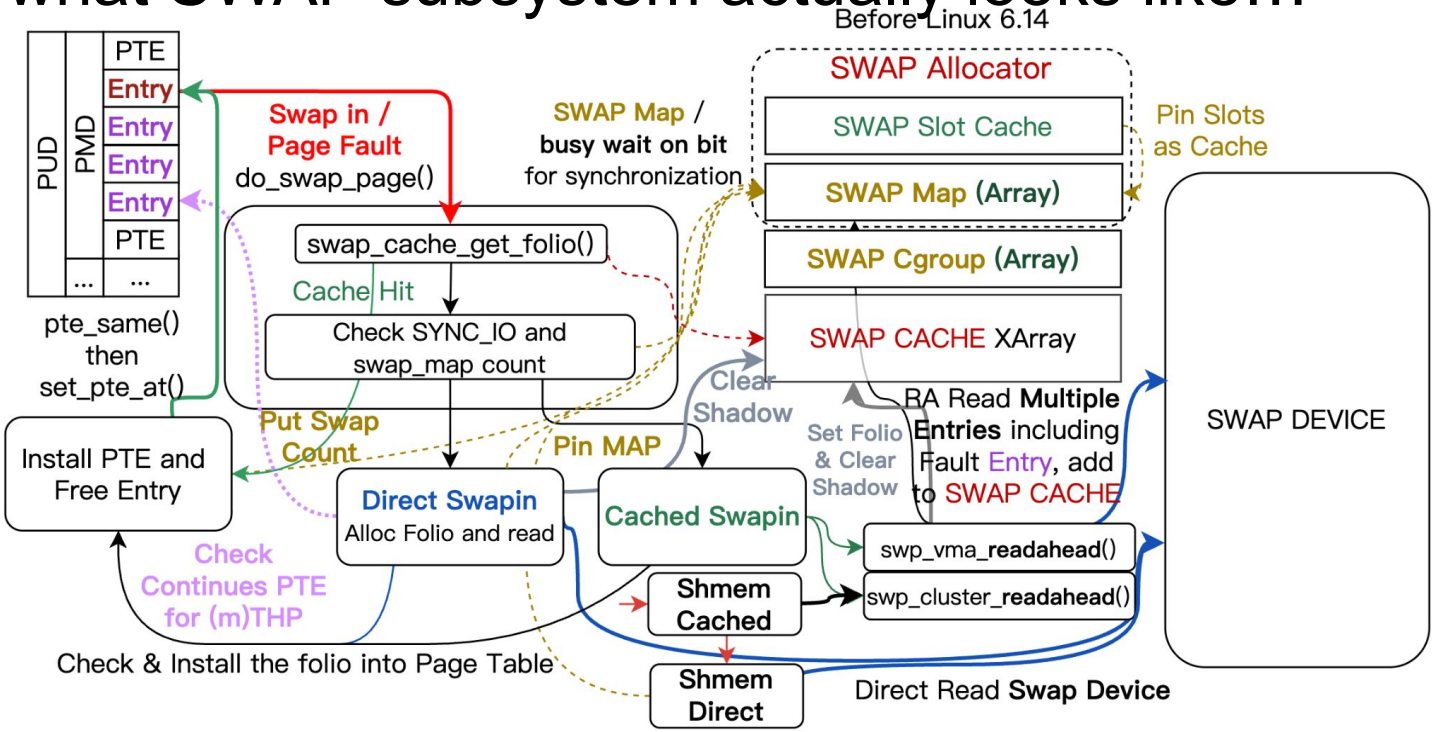
Shadow: An 8-byte, “timestamp”-like mark left in the map to track when the folio was evicted, used to refine the evaluation of memory hotness..

This is (not) what SWAP subsystem actually looks like…

Swap-in stack pre-6.14

Much effort went into optimizing it with more layers or shortpaths.

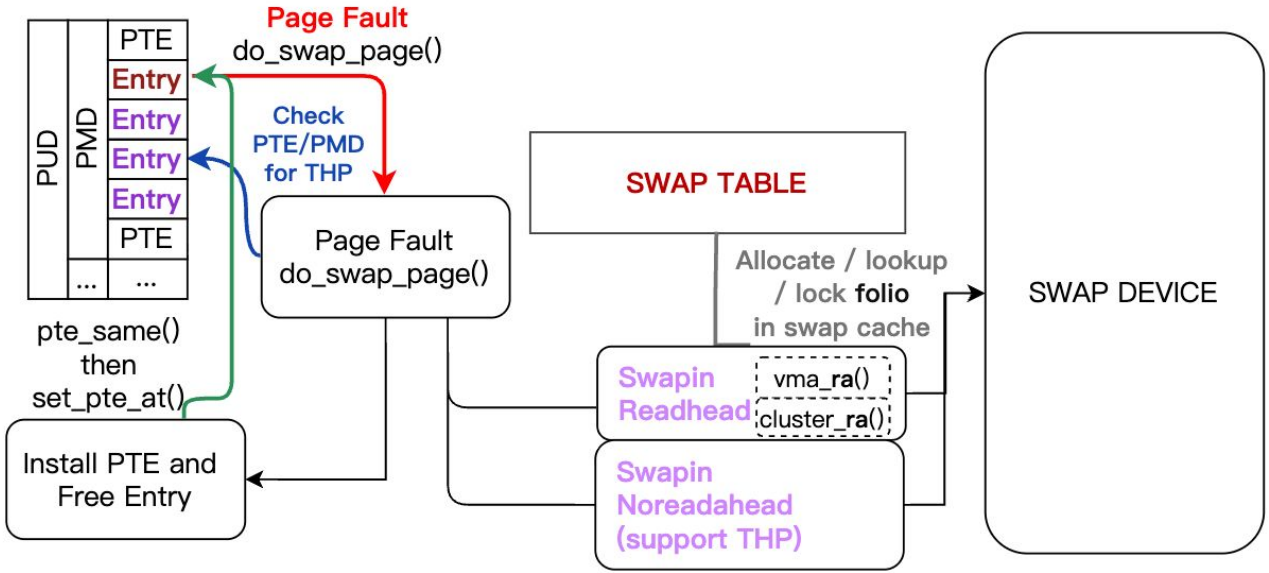
Led to **many** workarounds.



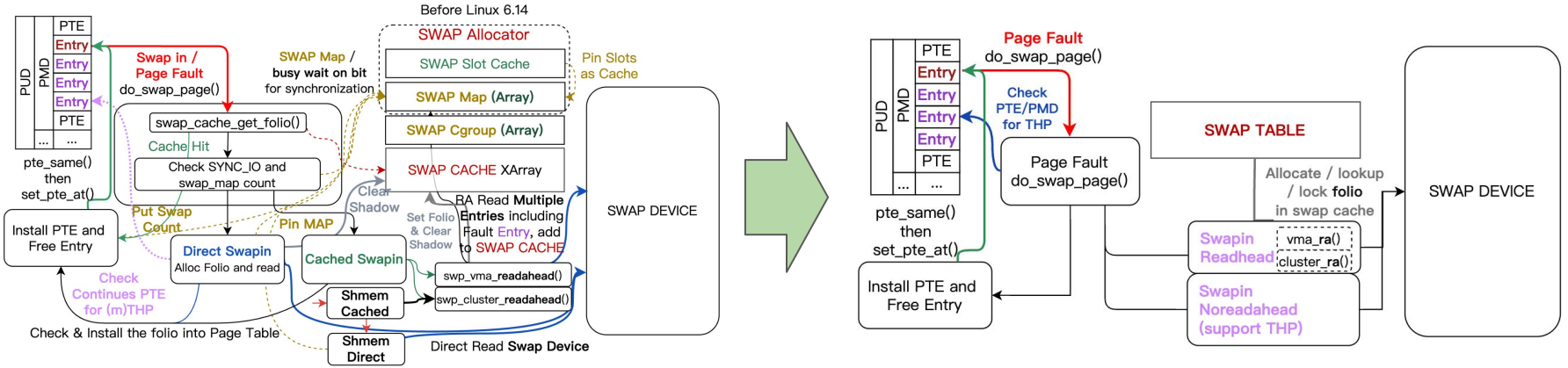
"All problems in computer science can be solved by another level of indirection, except for the problem of too many layers of indirection"

This is what swap in looks like now

Through simplification and modernization efforts, the subsystem is much cleaner and more efficient.



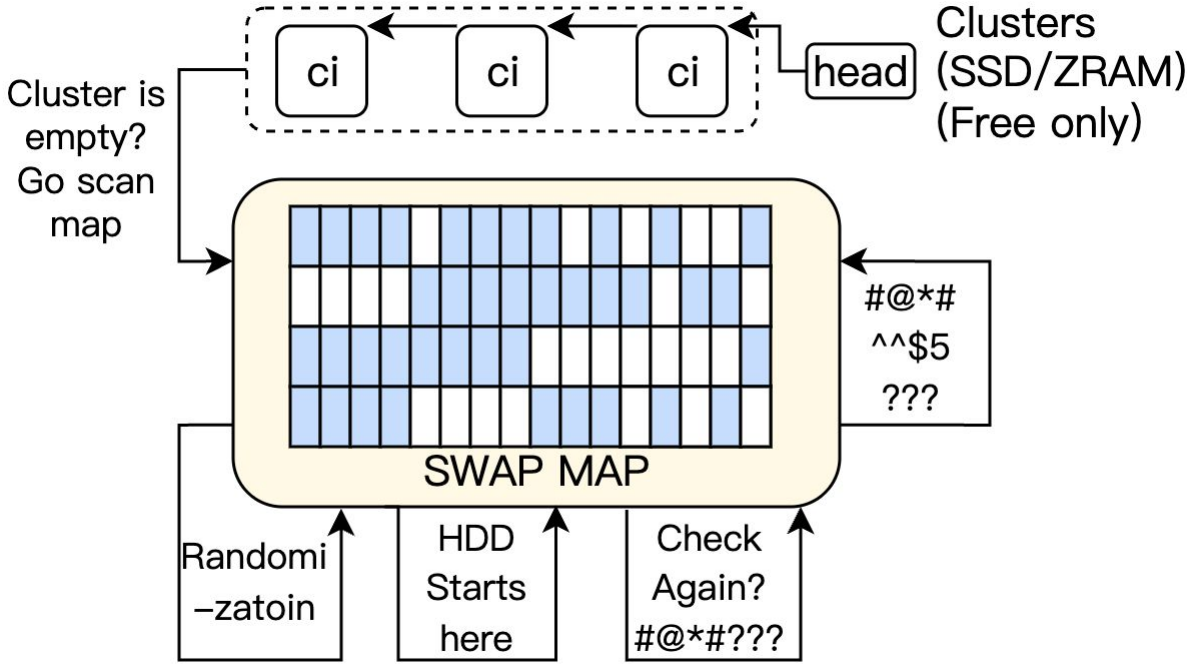
Modernizing the Swap Subsystem



https://lwn.net/Articles/1056405/

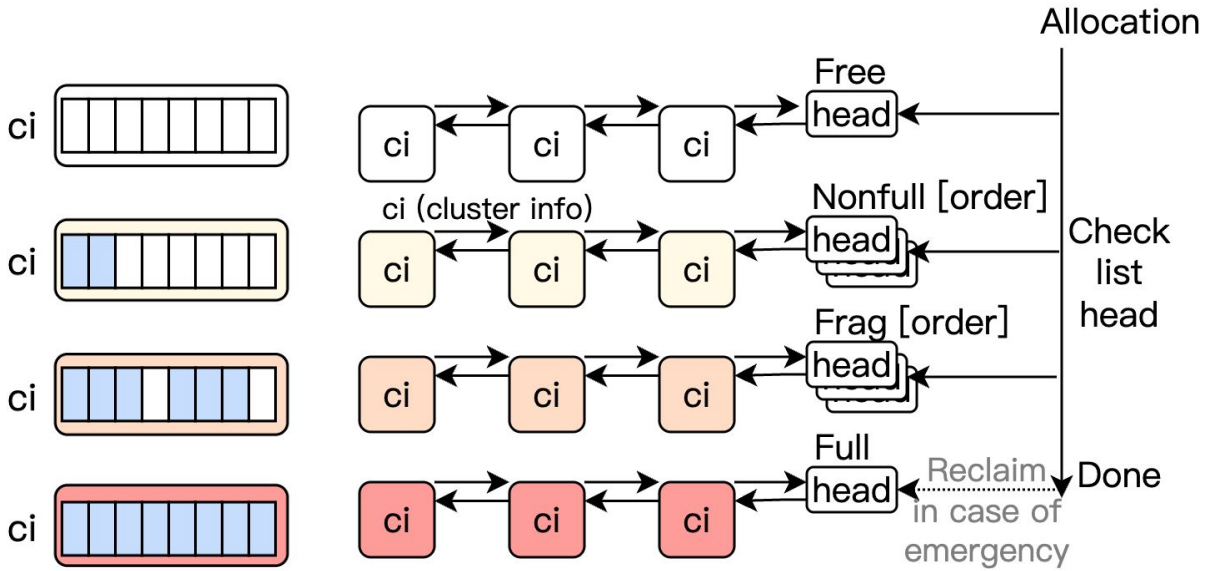
Modernizing the Swap Subsystem: Cluster Allocator

- First step: **swap allocator**
- The old allocator is built around the static swap map, which is also the core of the old swap infrastructure.
- Complex and performing poorly.
- **Optional clustering (2MB)**, with global scans and tangled logic (PCP randomization, SSD vs HDD).

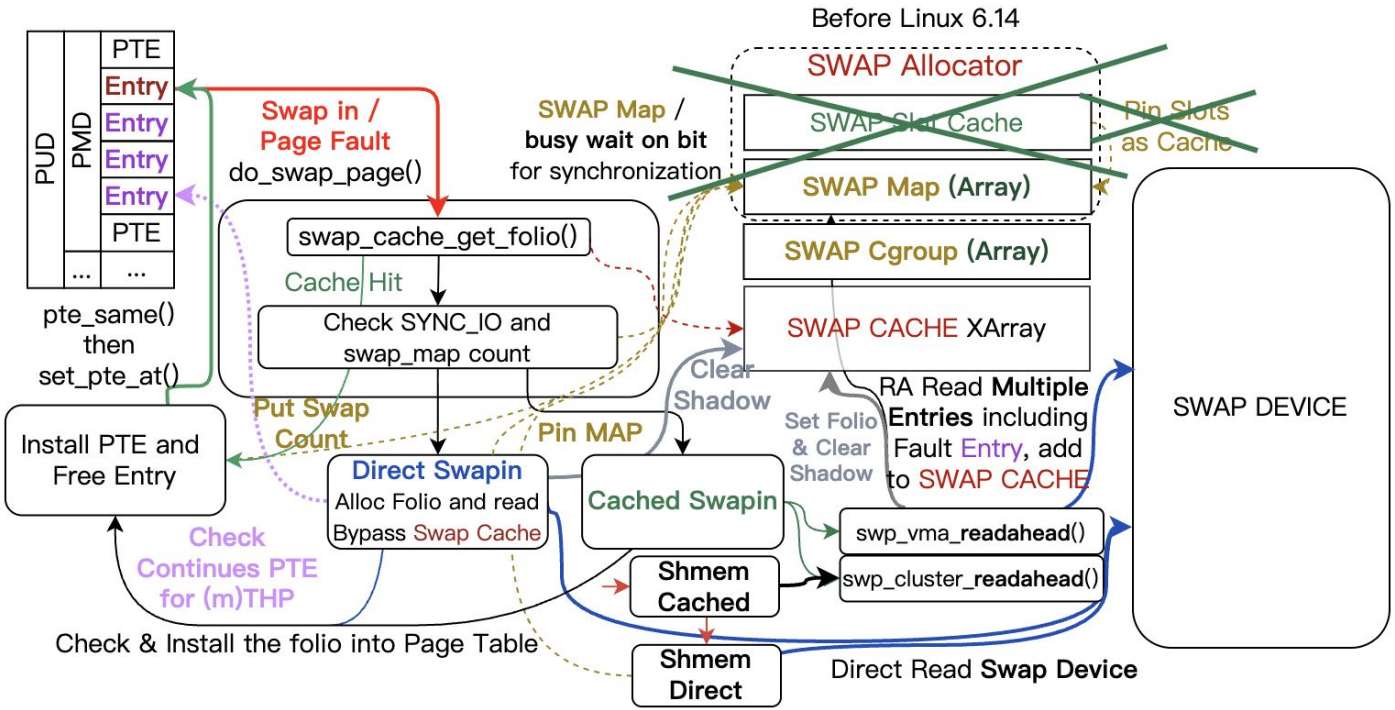


Modernizing the Swap Subsystem: Cluster Allocator

- **Enforce mandatory 2MB swap clusters**.
- All clusters are always on a list.
- Allocation is now just lookup of list heads.
- Eliminate global operations and rework locks to remove contention.

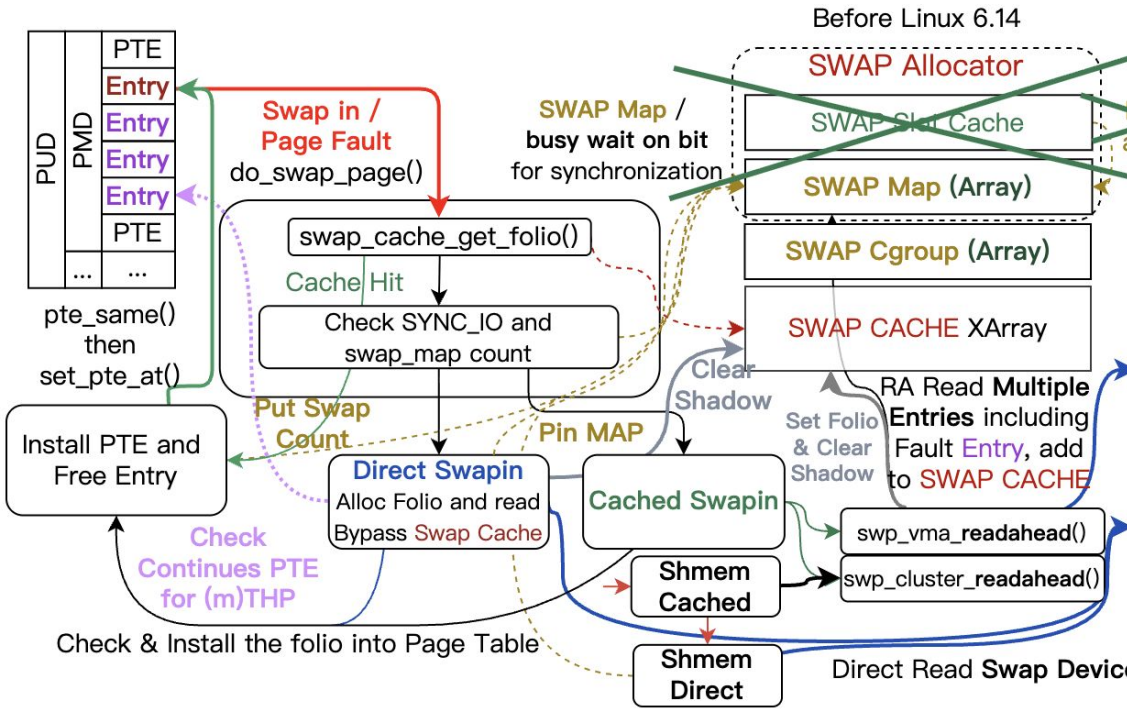


Modernizing the Swap Subsystem: Cluster Allocator



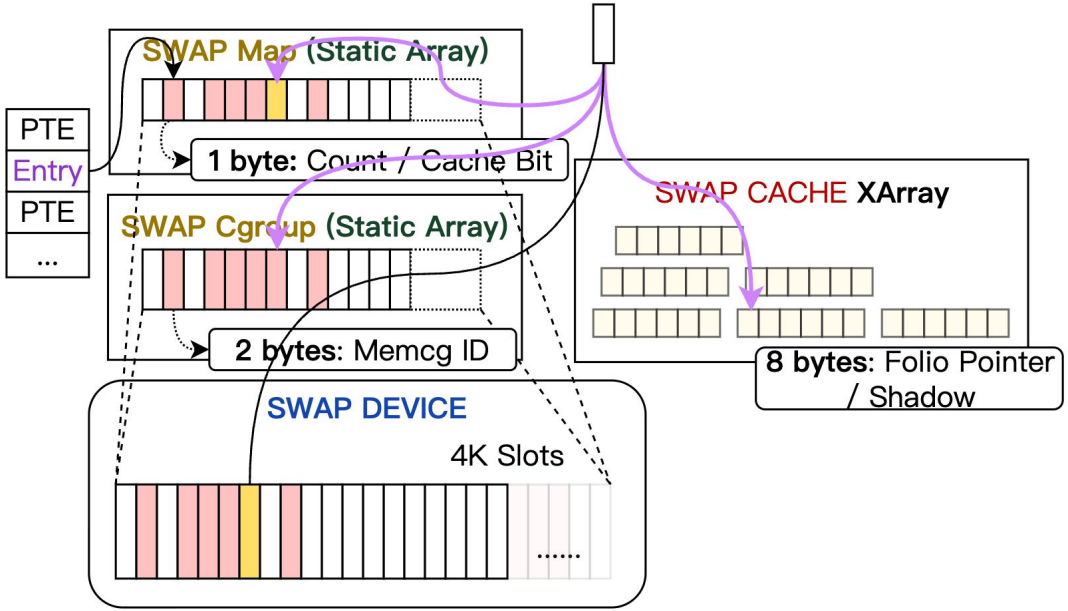
Modernizing the Swap Subsystem: Merging Metadata

- Look at the other parts…
- We tried various ways to merge or simplify things; for instance, many issues were caused by “swap cache bypassing”, so we really should remove that.
- This did not work out well, resulting in either performance drops or memory bloat.



Modernizing the Swap Subsystem: Merging Metadata

- The 1 + 2 + 8 bytes per slot metadata is the key.
- It is a hybrid of a static map and a dynamic xarray, with disparate data sources that are hard to manage.
- swap_map is hard to challenge (1 byte per slot).
- To improve it, we need to get rid of this burden and adopt an even more compact and dynamic format.



Swap Metadata: Introduce Swap Table

- One slot can only be in one of three status:
  - **Free** / **Used** (**uncached**, swapped out) / **Used** (**cached**) / **Special value** (Bad slot, can't use).
- One unsigned long (64 bits) is (almost) enough to represent everything we need.
- Encoding:

```
* NULL:     |---------------- 0 ---------------| - Free

* Shadow:   |SWAP_COUNT|Z|---- SHADOW_VAL ---|1| - Swapped-out (Uncached)

* PFN:      |SWAP_COUNT|Z|------ PFN -------|10| - Cached

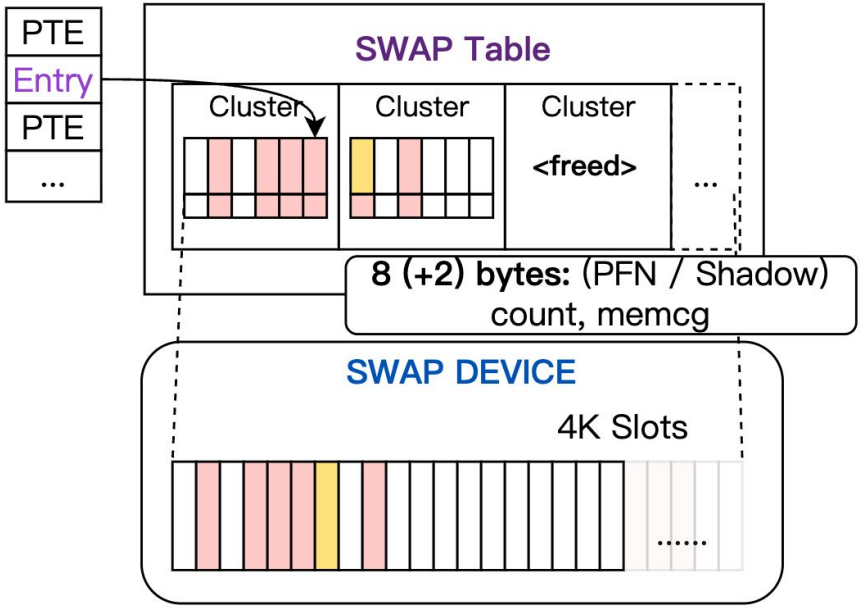
* Pointer:  |----------- Pointer ----------|100| - Reserved

* Bad:      |------------- 1 -------------|1000| - Bad slot
```

- Used and uncached slot: Shadow - the "timestamp" doesn't need 64 bits, can easily make space for 6 bits count field.
- Used and cached slot: store PFN (52 bits), cleanly engouth bits for count in upper bits.
- As we has made cluster the basic unit for swap management.
- Each cluster just use a array of unsigned long for both count and cache.

Swap Metadata: Introduce Swap Table

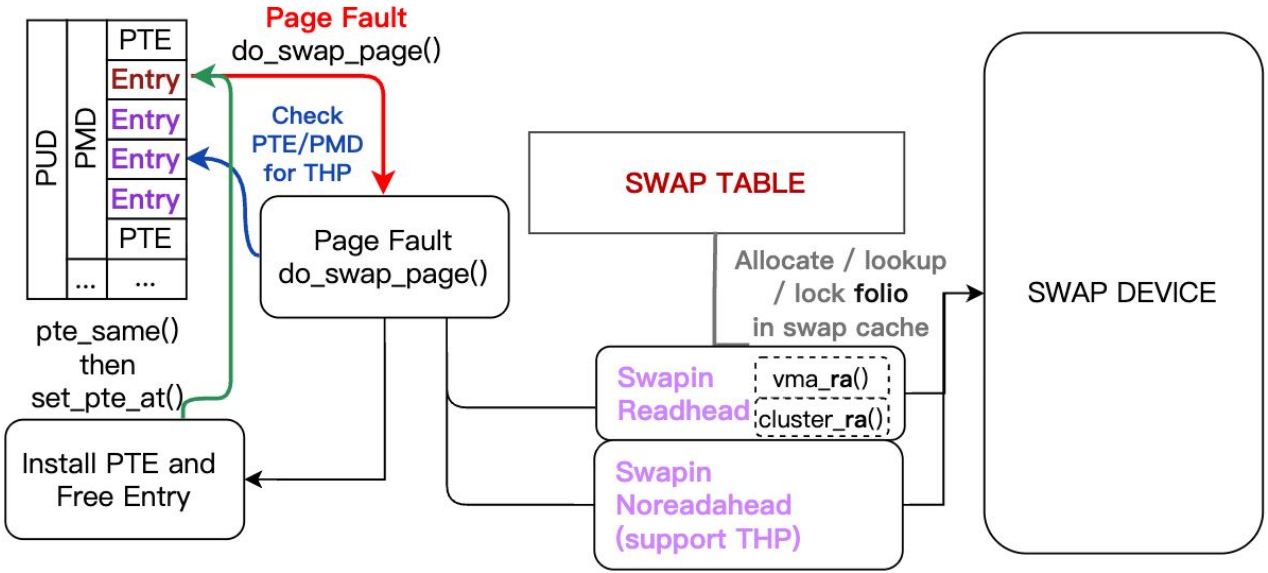
- Only in-use clusters need the table allocated, while free clusters can release it.
- Table size is exact 4K (8 x 512).
- Lower memory usage.
  - The 1 byte swap_map merged with the 8 bytes swap cache into one 8 bytes swap table entry.
- Better performance
  - More compact data source
- No more synchronization burden / hacks.



We also split the memcg array into per-cluster tables, achieving close to zero idle memory usage

And this is what swapin looks like now

- Operates entirely on a single, cluster-based data source.
- Eliminated legacy workarounds,
- Reduced maintenance overhead, lower memory usage, and improved performance.

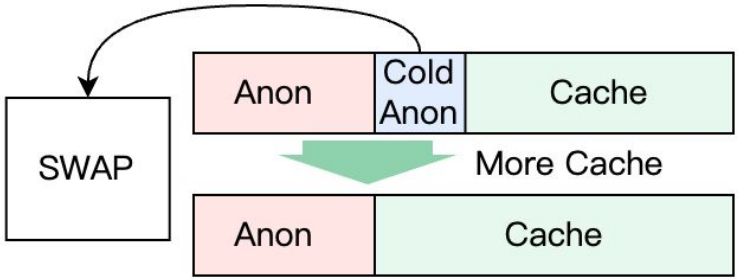


The kernel internal API is much cleaner too.

Swap is actually (more and more) beneficial for performance

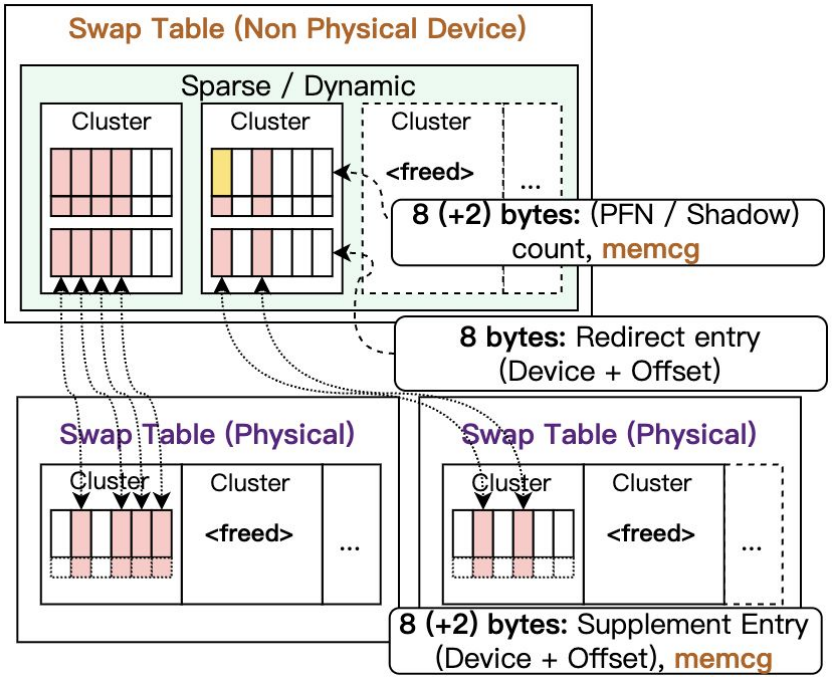
- **Swap had a bad reputation for a good reason.**
- Things change, as swap and storage become faster and faster.
- Memory compression provides free memory by trading CPU cycles for RAM.
- Your machine is very likely to have a lot of single-use cold anon memory.

- Enabling swap allows your kernel to cache more hot data
- Poor performance? Could be wrong eviction? we are working on it :)



Future Items for SWAP

- Better Readahead & Better THP Swap-in / Swap-out
  - How can swap readahead be improved?
  - Avoiding page faults before they occur?
- Tiering
- Resizing
- Balancing
- Migration
- IO Batching
- **Better non-physical Swap**
- **Finding the right folio to swap out**



Finding the right folio to be swapped - Memory Reclaim

- Memory reclaim is a broader topic, beyond just swap.
- There are roughly two types of user space memory:
  - **Anon** folio -> Swap backed -> Swapout
  - **File** folio -> File backed -> Page Cache -> Drop (if clean)
  - Shmem (Swap backed) / other
- Free memory is wasted memory, and the page cache stays until memory is under pressure.
- Goal: Finding the cold part of RAM to evict when under pressure.
  - Swapped (Anon & Shmem) or dropped (File).
    - Anon memory could be colder, and either type of folio needs a refault (I/O) to be brought back.
  - Writeback is a bit different, not part of the topic.
  - Kernel memory is also a bit different, not part of the topic.

```
root@POTATO:~# cat /proc/meminfo
MemTotal:       65168928 kB
MemFree:         1176944 kB
MemAvailable:   28059076 kB
Buffers:            5376 kB
Cached:         29469972 kB
SwapCached:       928056 kB
Active:         40435788 kB
Inactive:       17057508 kB
Active(anon):   23554120 kB
Inactive(anon):  6731696 kB
Active(file):   16881668 kB
Inactive(file): 10325812 kB
SwapTotal:      67108860 kB
SwapFree:       64885880 kB
Dirty:             29168 kB
Writeback:          2184 kB
AnonPages:      28000872 kB
Mapped:          2985484 kB
Shmem:           2430388 kB
KReclaimable:     954744 kB
Slab:            1656404 kB
```

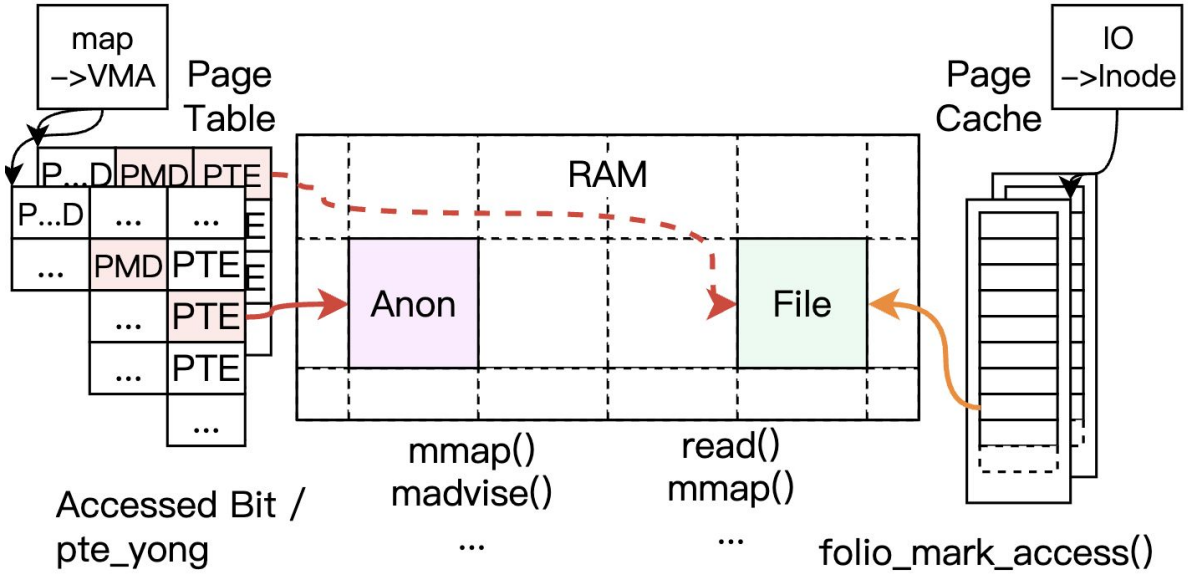
Userspace Memory Reclaim

To figure out which folios need to be reclaimed, the memory management subsystem needs to:

- Collect hotness information.
- Evaluate the hotness information.
- Evict cold ones efficiently when under pressure..

Hotness Info, over-simplified

- Roughly two types of memory.
  - Mapped / Unmapped
- For mapped (anon / file)
  - PTE Access Bit
  - Set by CPU on access
- For unmapped (file)
  - folio_mark_accessed()
  - Called by kernel proactively
- Temporal Locality (LRU)
  - Folio allocated recently is more likely to be used again.
- Refault
  - Evicted folios leave a "timestamp" at their original mapping
- Other (madv, fadv…)



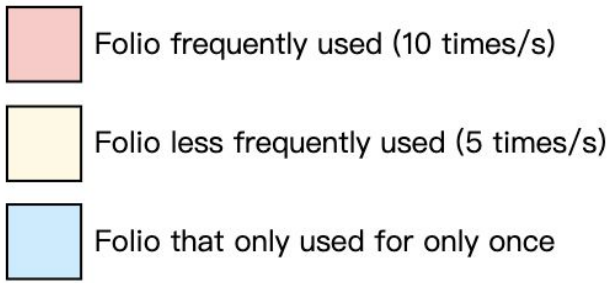
Challenges Evaluating the Hotness Info

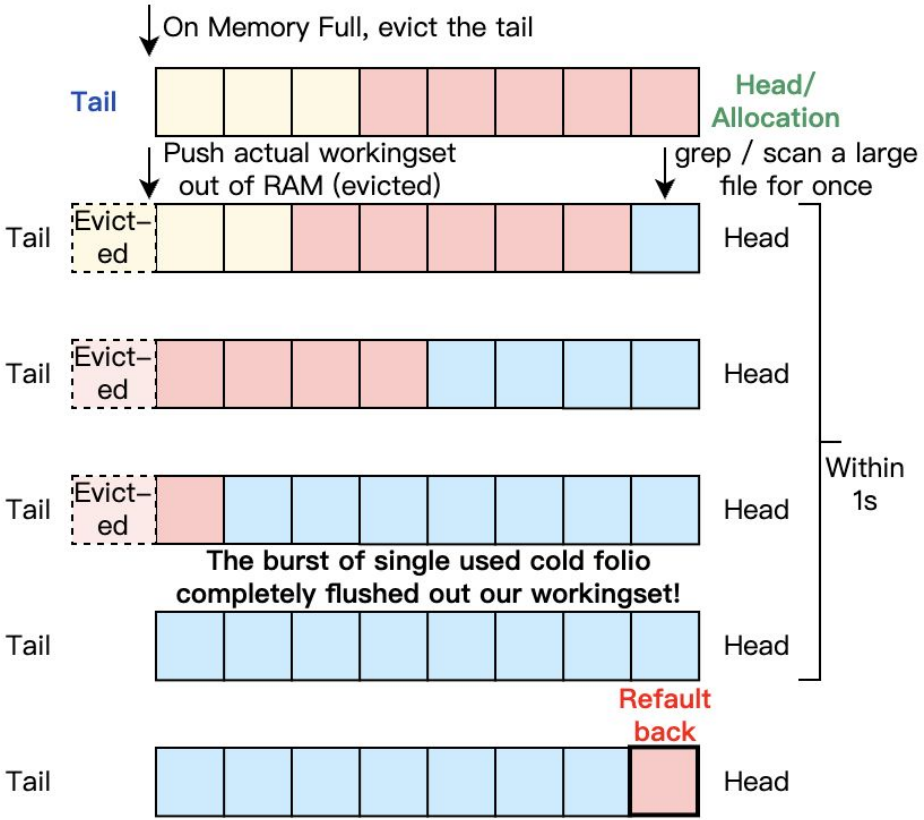
- PTE Access
  - It’s sticky: It’s sticky: one PTE can be accessed multiple times, and the access bit is a boolean.
  - It’s costly to collect. Page table is really good at forward lookup (VA -> PTE -> folio, Cache, Batching), more costly at reverse lookup by **RMAP** (folio -> PTE, rmap walk one by one), and kernel need to modify the page table to reset it.
- folio_mark_accessed()
  - It sits in very busy path, needs to be as fast as possible.

Challenges Evaluating Hotness Information (LRU)

- Cold Cache Pollution

  A burst of single-use cold cache can push hot cache out of RAM when using a simple LRU



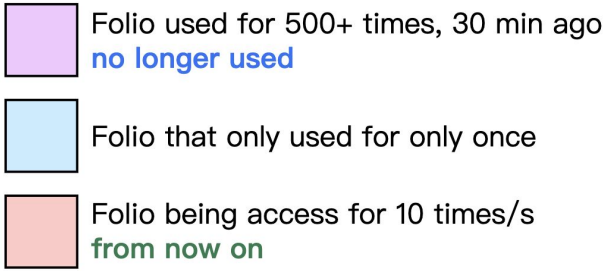


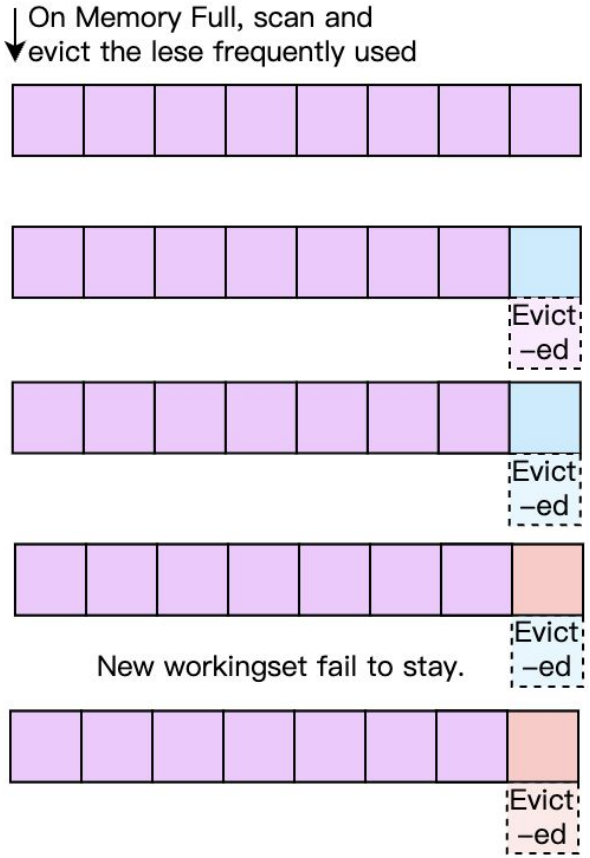
Challenges Evaluating Hotness Information (LFU)

- Stale Cache Pollution

  If we evict less-used folios instead (LFU), cold cache pollution is fixed, but historically hot folios stay too long, and new working sets fail to be retained.

- Scanning and sorting are costly.

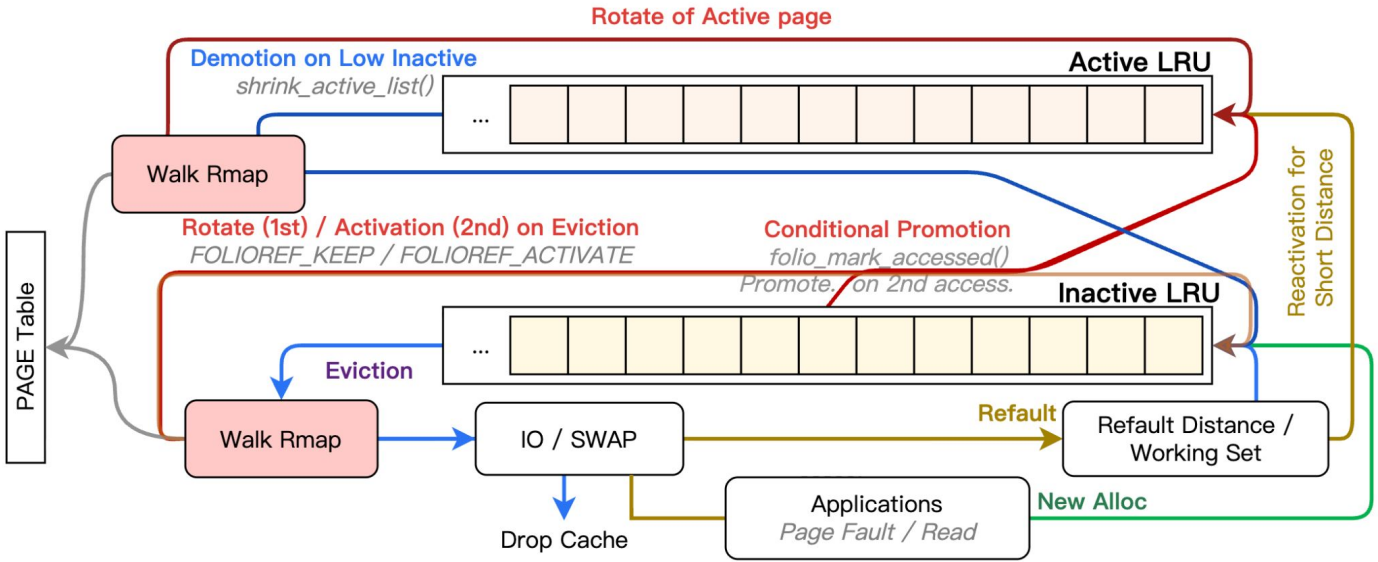




CLRU - Kernel's Classical Active/Inactive LRU

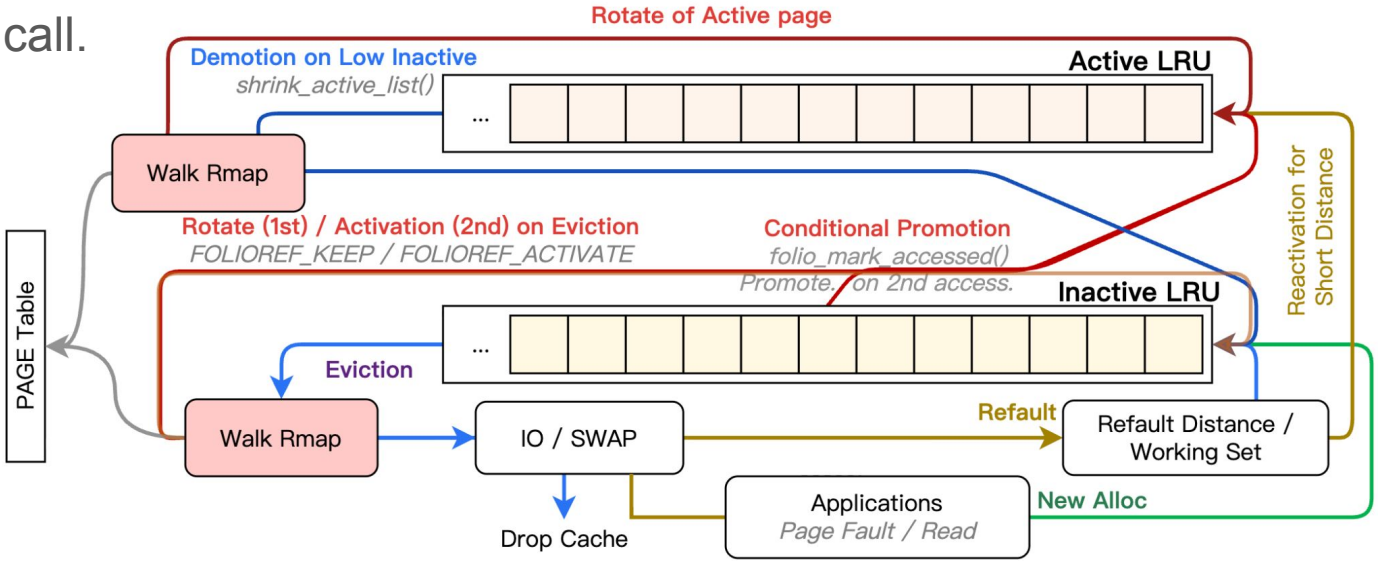
- "Classical LRU", as it has been battle-tested in the kernel for decades
- Two LRU lists, an active list and an inactive list, distinguish the **working set**.

- Single-use folios stay in the inactive list
- Requires two accesses to promote a folio to active list.



CLRU - Kernel's Classical Active/Inactive LRU

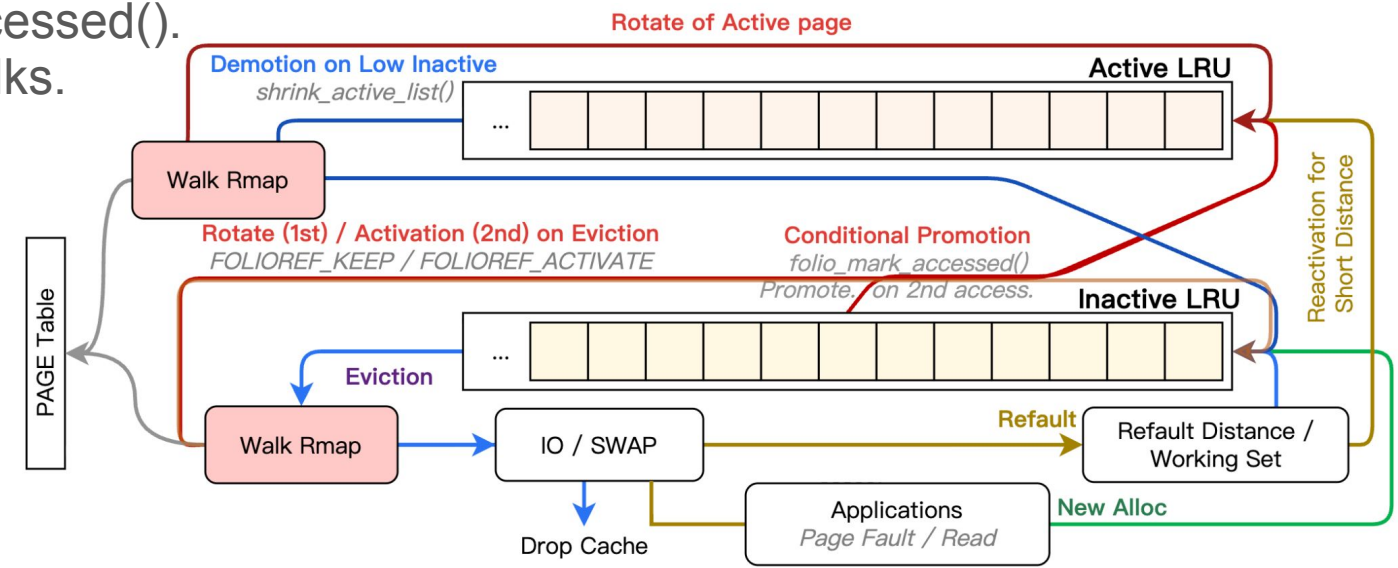
- All folios start at the head of the inactive list..
- **folio_mark_accessed**(): -> PG_referenced on first call -> move to active head on second call.
- **PTE access:** via rmap walk and reset on eviction or demotion.
- First PTE access: rotate it.
- Second PTE access: promote it.



- Shrinks the active list when the inactive list becomes too short.

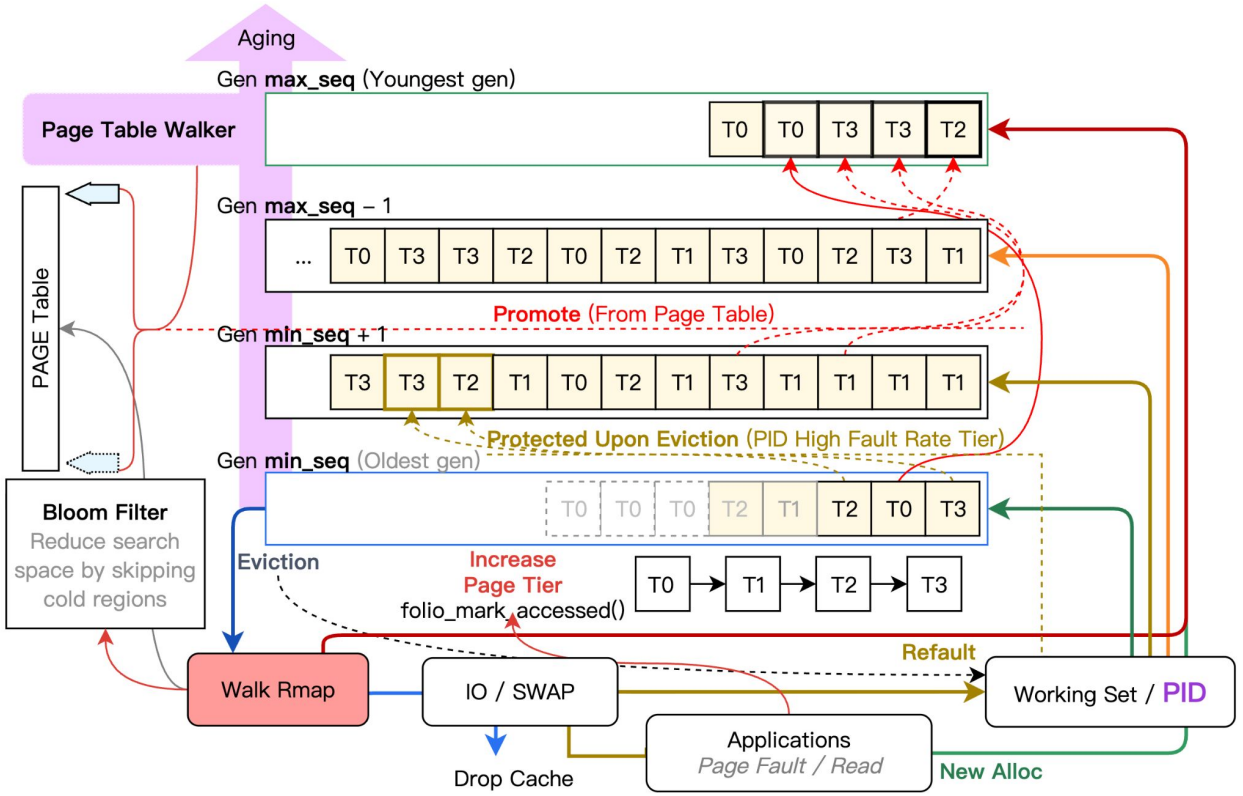
CLRU - Kernel's Classical Active/Inactive LRU

- Problems:
- Both demotion and deactivation require the LRU lock, as does folio_mark_accessed().
- Two Rmap Walks.
- Only two levels of hotness.



MGLRU

- Answers the challenges in a different way.
- 4 Gens, 4 Tiers
- 3 (gen) + 2 (tier) bits in folio flags
- Page Table Walker
- Lazy Promotion
- Rmap Lookaround
- Bloom filter
- PID feedback
- It’s a framework.



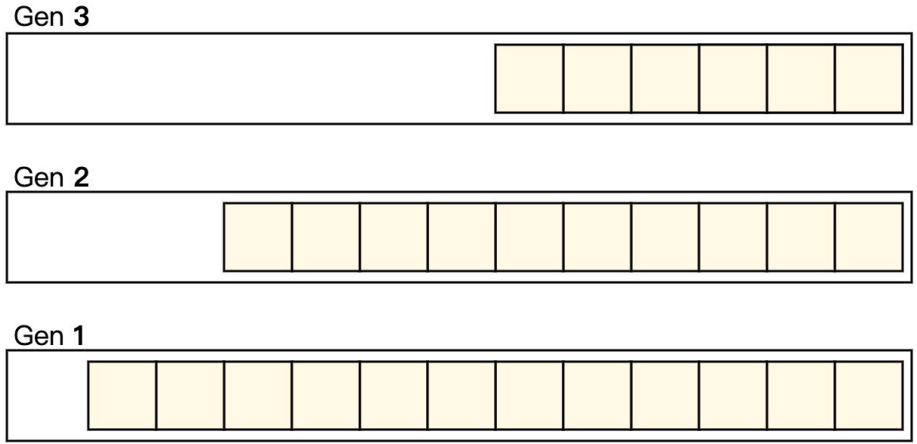
MGLRU - How it works - Aging

The number of gens on a MGLRU-enabled system is dynamic

(MIN_NR_GEN = 2, MAX_NR_GEN = 4).

MGLRU tries to add add newly allocated folios to one of the gen in a heuristic way.

(Ignoring certain details here as they are constantly changing)

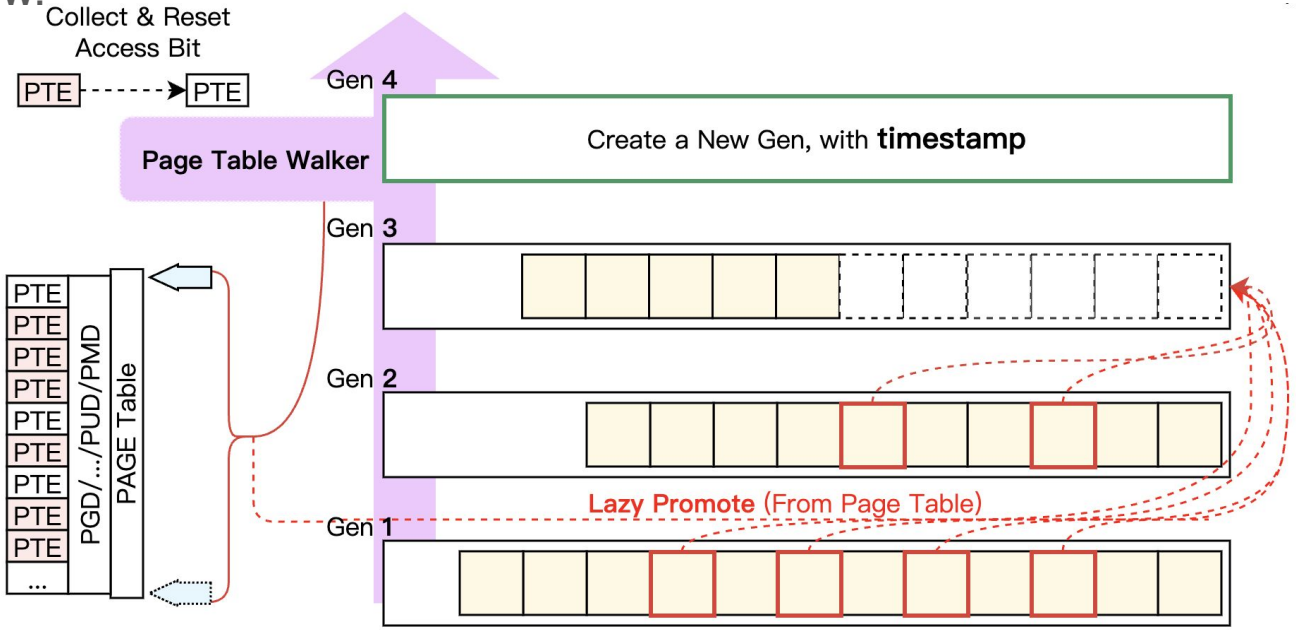


MGLRU - How it works - Aging

Under memory pressure, MGLRU may starts a process called “Aging” when gens number is low.

1. Walks the page table directly. Batch collect & reset, “lazy promote” (flags mark only, lockless) folio to newest gen. (also on 2nd access).

2. Populate a newer gen (with **timestamp embedded**) to be used later. 4 Gens now.

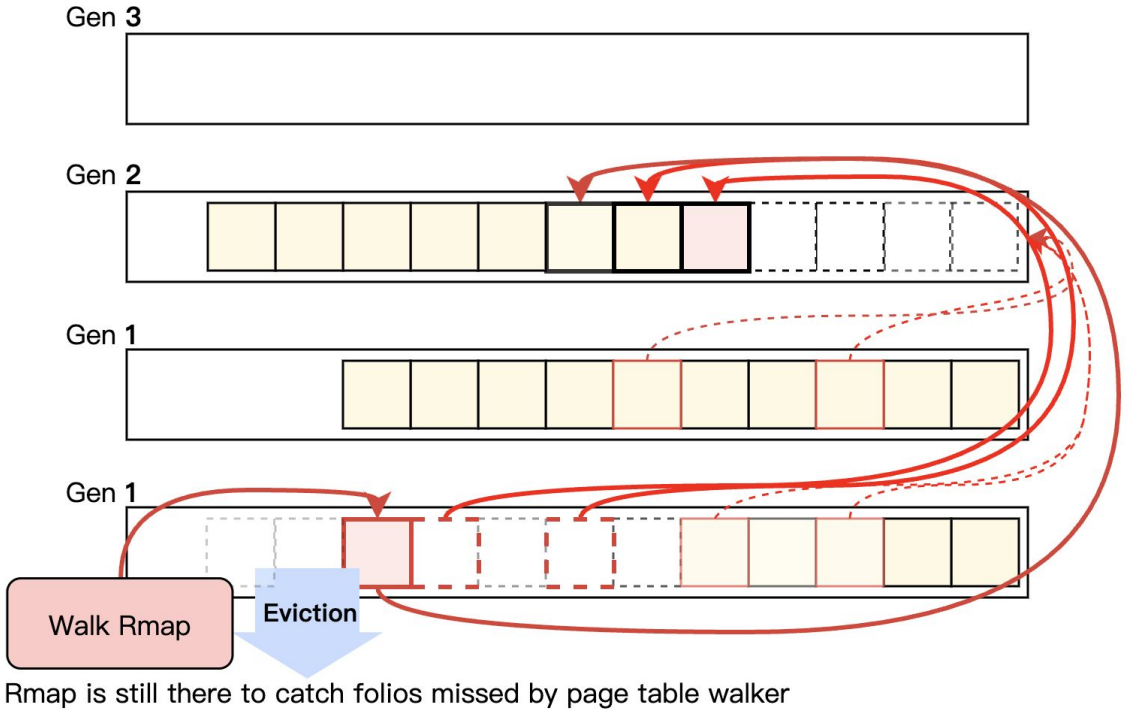


MGLRU - How it works - Aging

Eviction just evict folios on the oldest gen tail.

Lazy promoted folios will be moved directly by eviction: Sees the folio marked as belonging to a newer gen -> don’t evict it, just move it.

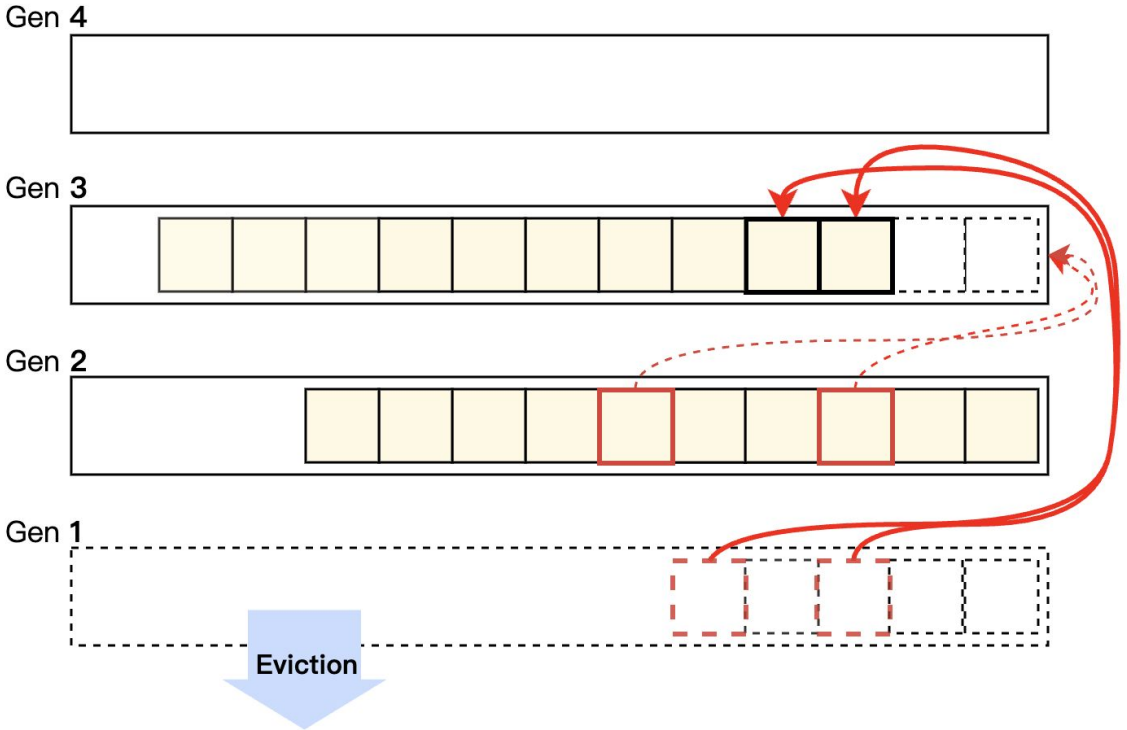
Rmap walk still there as supplement for page table walker, but skipped for already lazily promoted folios.



Rmap is still there to catch folios missed by page table walker

MGLRU - How it works - Aging

As eviction goes on, a whole generation is dained…



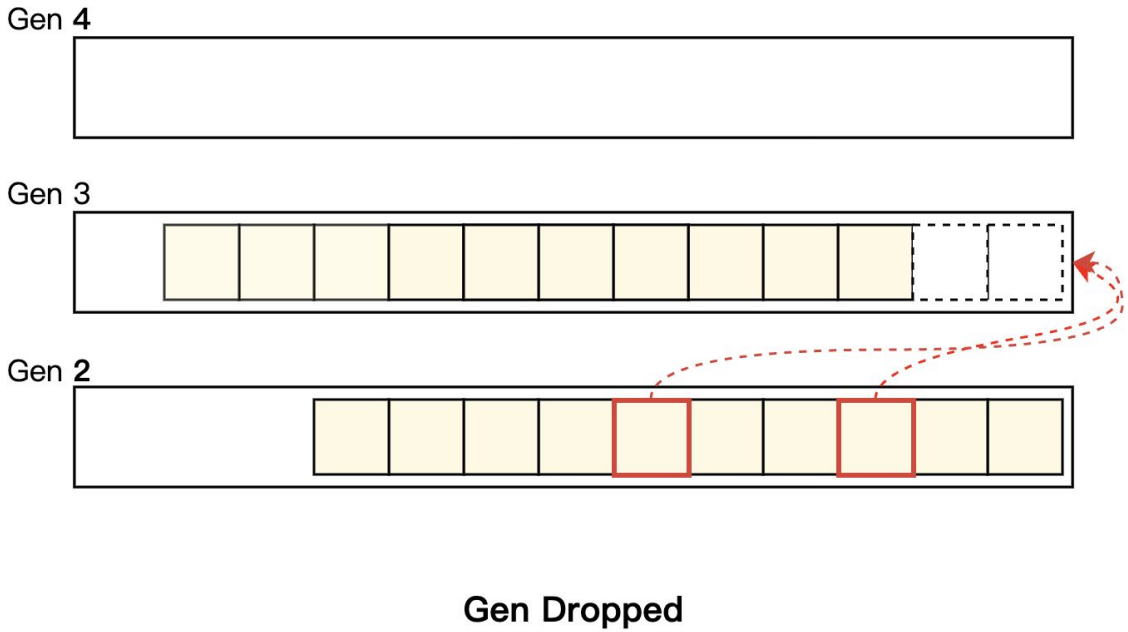
MGLRU - How it works - Aging

That generation is **gone**.

Dropping the gen is basically zero cost, no list move movement involved.

MGLRU uses a sliding window for gens.

3 gens again.

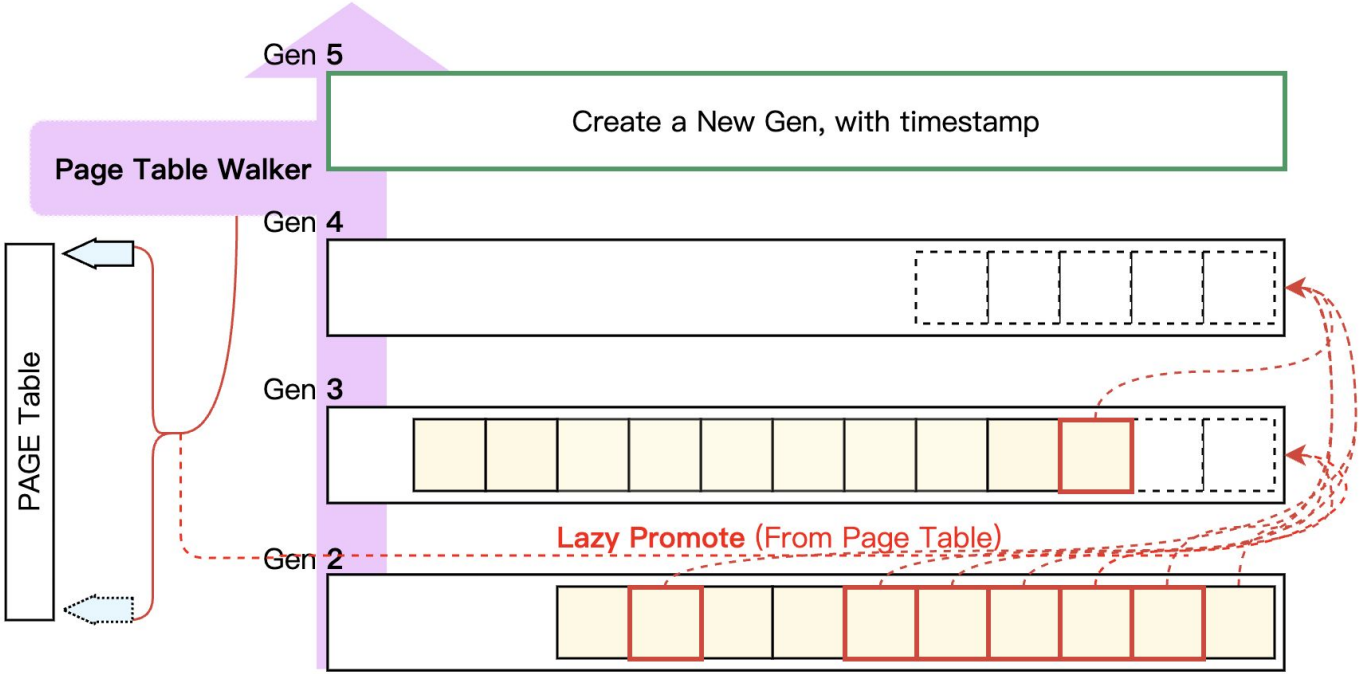


MGLRU - How it works - Aging

When under pressure again, repeat the aging process.

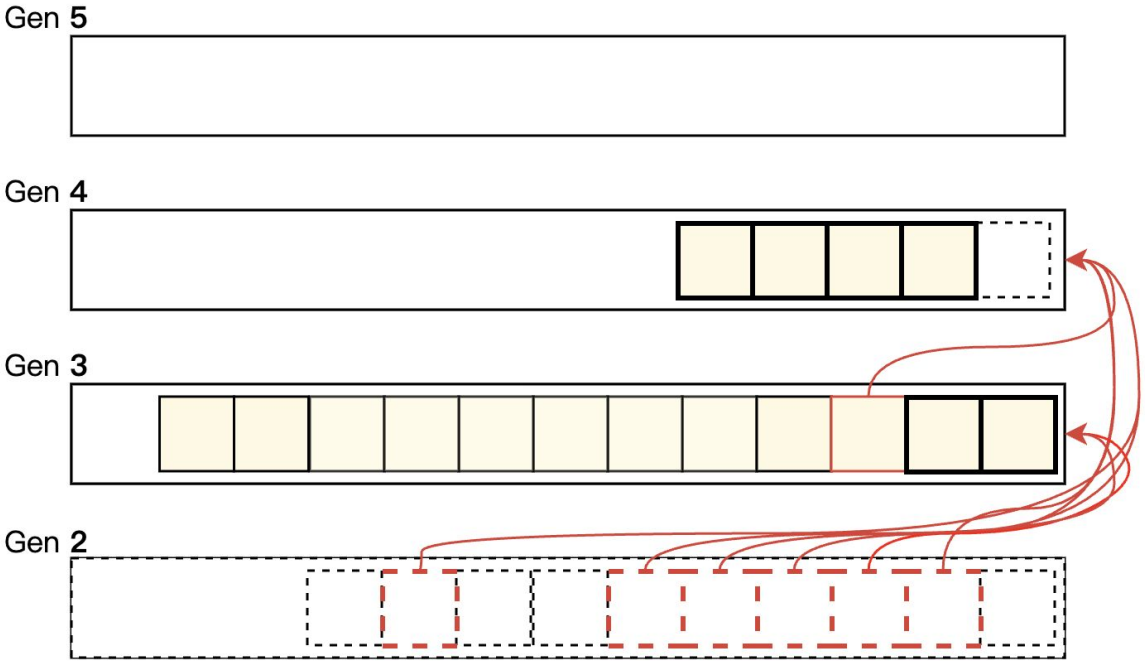
Batch collect & reset & lazy promote accessed folios from page table to current latest gen (gen 5)

Then create a new gen.



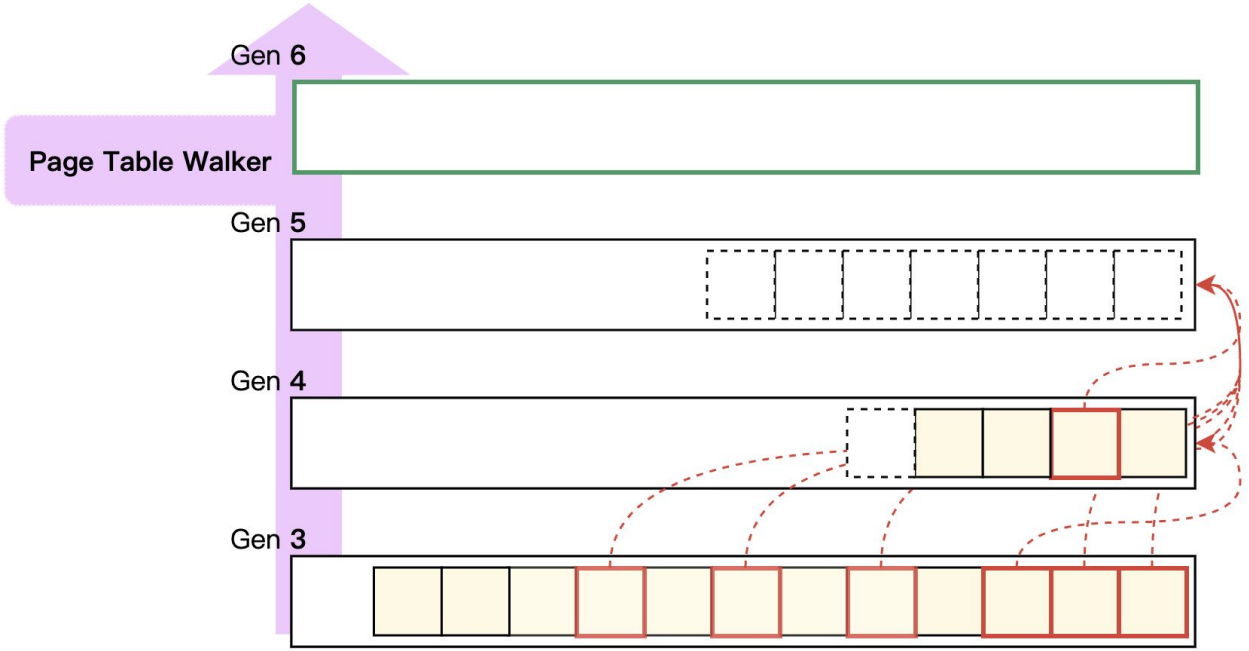
MGLRU - How it works - Aging

And evictions keep draining and dropping oldest gen.



MGLRU - How it works - Aging

And aging keep aging and promoting folios into newer gens.

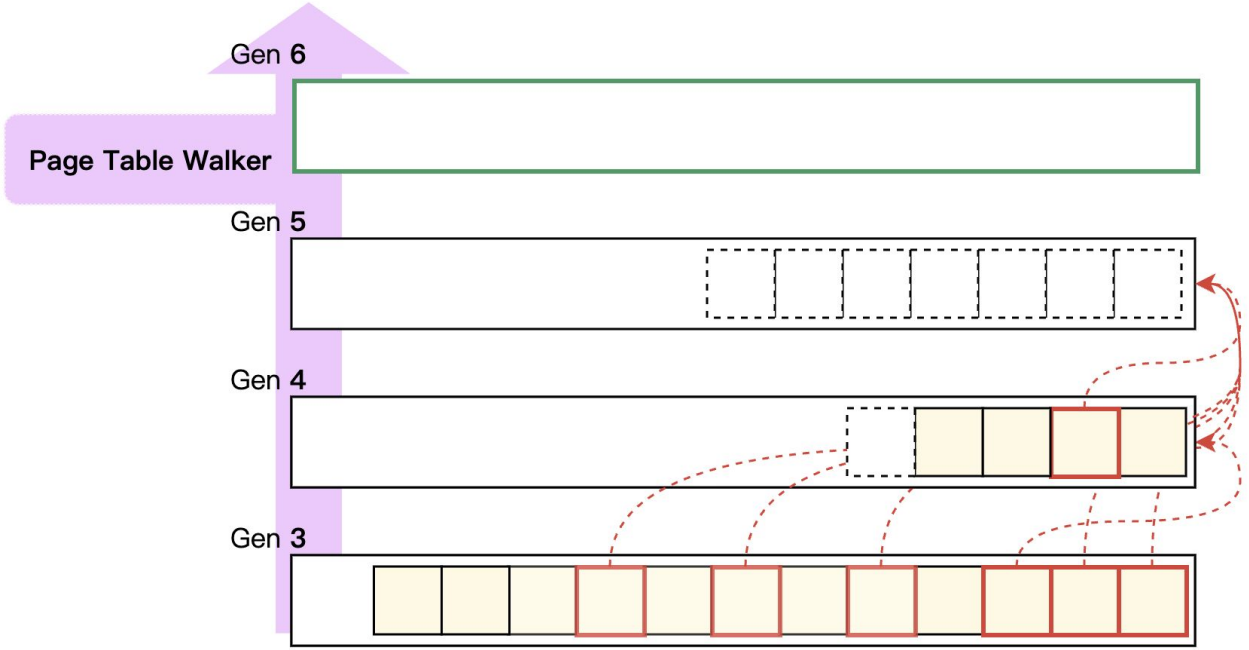


MGLRU - How it works - Aging

All the folios is being:

Promoted by aging / page table walker.

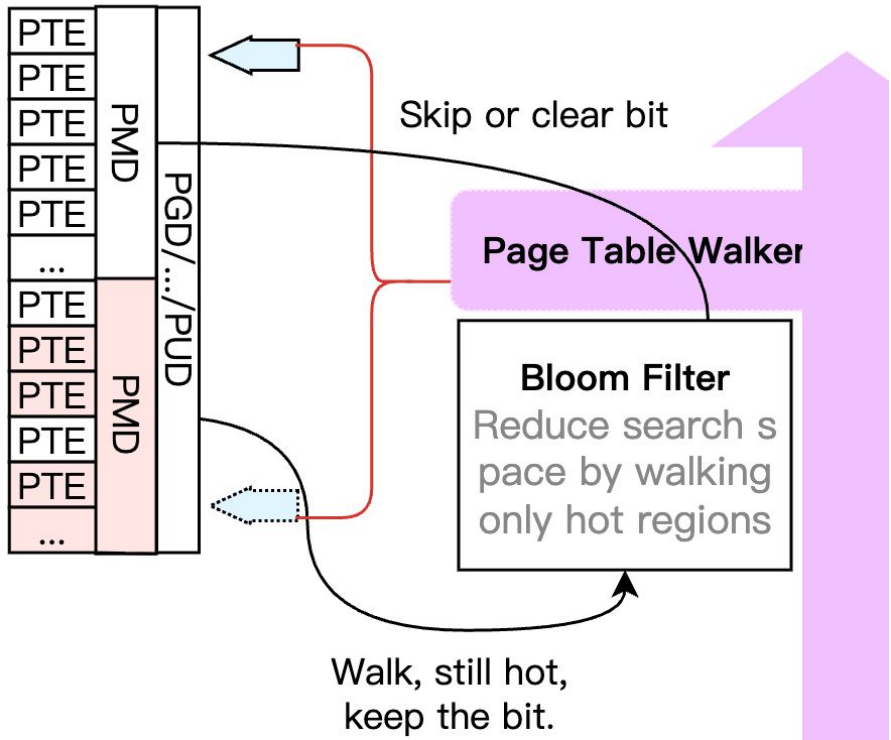
Evicted and gens dropped.



MGLRU - How it works - Aging

Aging could be slow? VA space is huge.

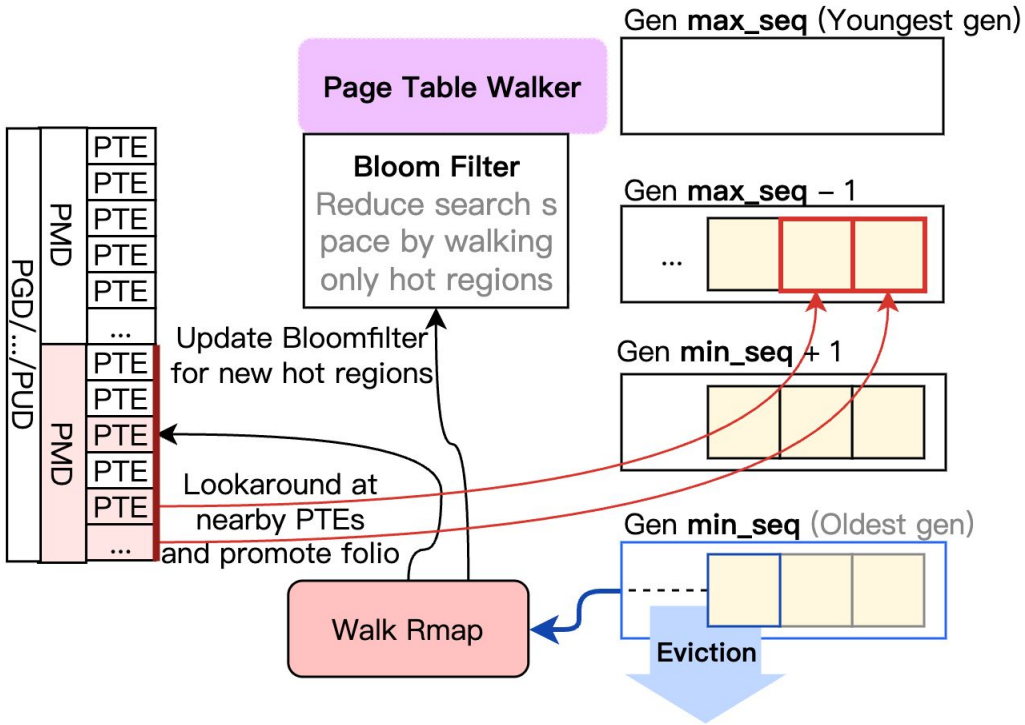
- Aging is lazy, it may defer aging unless there is only two gen left.
- Aging uses a **Bloom filter** to identify hot regions and only walks those hot regions.
- Aging updates the bloomfilter in case the region turns cold.
- If walk found that a region is still hot, carry related bits in bloomfilter.
- If walk found that a region is cold, drop related bits in bloomfilter.



MGLRU - How it works - Aging

Rmap is still there for several reasons:

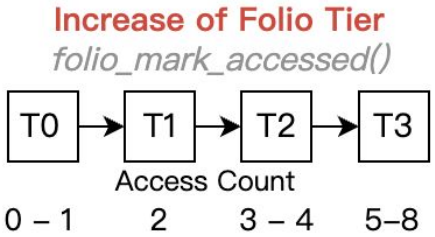
- **Update bloomfilter** for the new or missed hot regions.
- When looking at the PTE of a folio, it also does a **lookaround** to promote all nearby PTE folios (within a PMD), following **spatial locality**

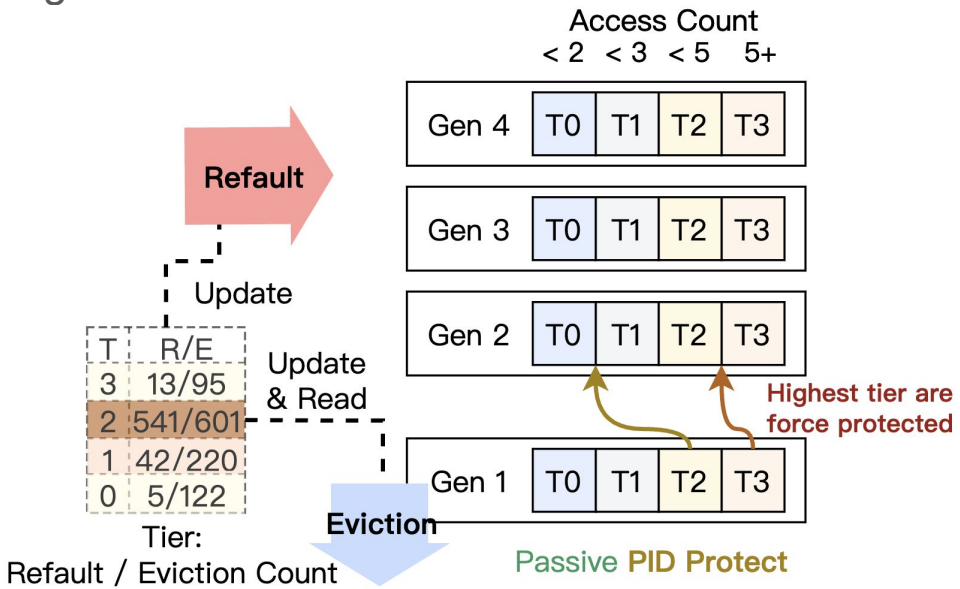


MGLRU - How it works - Tiering

Mapped folios are well protected, how about **folio_mark_accessed**()?

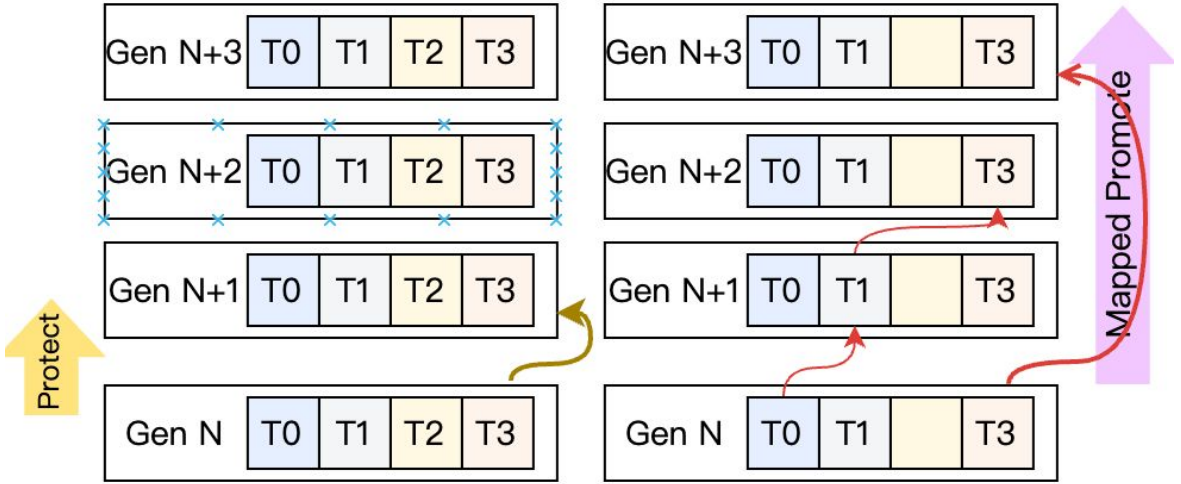
- Tiers is used to **protect** them. Folios catagrized as tiers.
- Kernel is aware of **refault**. Folios comming back after evicted, called a refault. (through shadow).
- If a tier has a high refault rate (refault / eviction), protect them **on eviction** (**bump by one gen**).
- Refault rate is soft reseted on aging.
- Called PID protection by MGLRU.





MGLRU - Problem of Tiering / folio_mark_accessed()

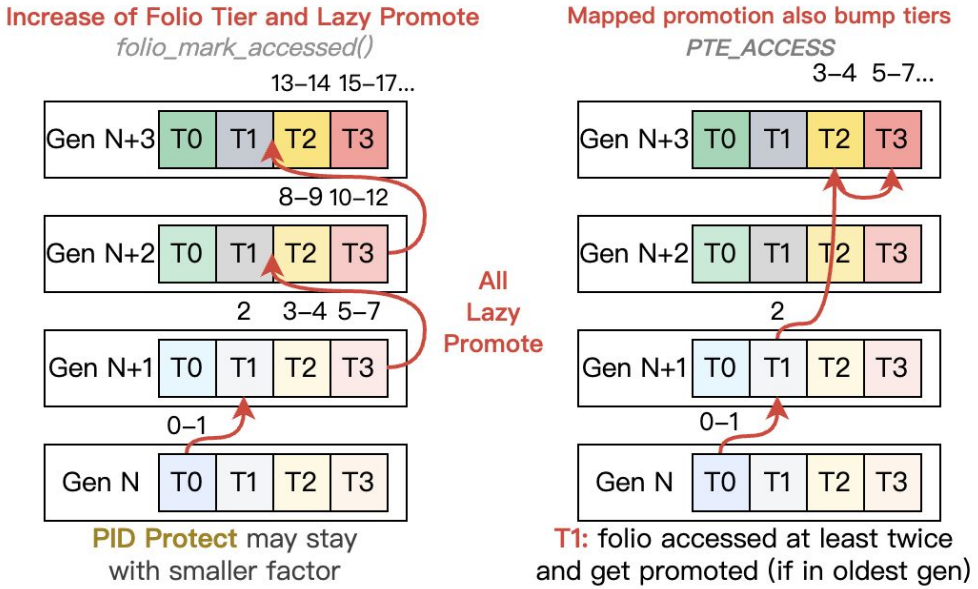
- Unmapped file cache is very common, PID protection is their only promotion:
  - It bumps unmapped folios by at most one generation, but bumps mapped folios to the latest generation, leading to the over-reclaiming of file folios
  - This breaks the LRU rule and causes stale cache pollution. (hotness info is evluated late)
  - Folios accessed for 8+ times can no longer be distinguished from each others
  - Doesn't apply well for anon / mapped folios. Page table walker is not making good use of tiers.



MGLRU-FG - Extend The Tier, Combine with Frequency

Instead of passive using passive PID, **apply lazy proactive promotion to folio_mark_accessed() / tiers too**. And make tiering a unified hotness baseline.

- Cold cache pollution:
  - Cold cache stay at lower gen.
  - Never effect newer gens.

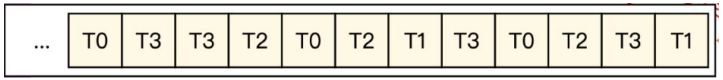


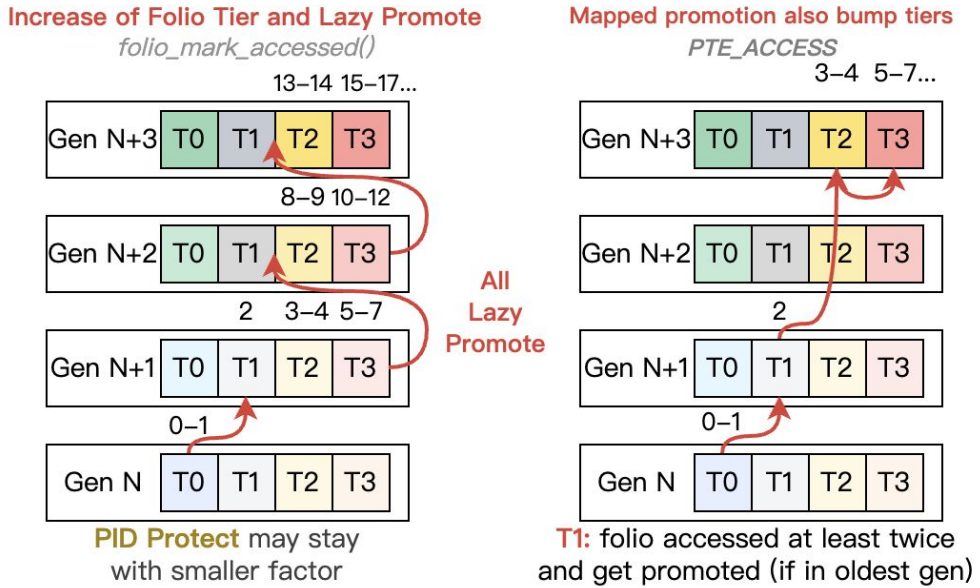
Initial test is showing really good results, RFC posted.

MGLRU-FG - Extend The Tier, Combine with Frequency

Instead of passive using passive PID, **apply lazy proactive promotion to folio_mark_accessed() / tiers too**. And make tiering a unified hotness baseline.

- Hot cache pollution:
  - Aging downgrades all folios with zero cost.
  - Folio eviction are purely LRU-ordered. New promoted access / allocated folios are always last to reclaim.





Initial test is showing really good results, RFC posted.

MGLRU - A promising framework, with new problems too

- A lot of problems can be solved by cusntomized policies
- Not really fesible to develop a custom policy for every application.
- The default behavior can still be be improved.
- cat **/sys/kernel/debug/lru_gen_full**

Memory Reclaim’s other parts & issues

- Swappiness - Balancing the Reclaim of two types
  - Anon and File are separate in to LRU lists
  - CLRU: Reclaim two list in a balanced way, based on swappiness.
  - MGLRU: Synchronized gen for two types, reclaim the oldest gen with lowest refault rate.
    - Swappiness improvements incoming!
- Avoid Thrashing - Wake up the OOM Killer in time
  - MGLRU - TTL using the timestamp.
- Refault Distance
  - Uses the “timestamp” in shadow to supplement the hotness info (how long / far a folio is evicted out of the memory).
  - MGLRU, insteasd, uses gen number to estimate that.
  - But Refault distance could be applied to MGLRU as well (RFC posted).
- Throttling
- Cgroup Balancing

With better swap and better memory reclaim.

- Free memory is wasted memory.
- **Cold memory is wasted memory.**
  - **Archive more using less memory!**
- Swap and memory reclaim help you to archive more with less memory.
- Enabling swap allows your kernel to swap out the cold anon folios, and do more caching to reduce IO.
  - Faster git log / grep / etc.
- Not perfect, a lot of problems to be solved, but we are getting better and better at it by improving the default implement.

QA

Thank You!

Backup Slides

Hotness

- Mapped folios (Anon folios, mmap’ed file folios, etc.)
  - PTE access bit
- File folios (read())
  - folio_mark_accessed()
- MADV
  - WILL NEED
  - DONT NEED
- Ways to collect and make use of these “hotness source”
  - folio_mark_accesses() - Direct Function Call (Classical LRU, MGLRU)
  - PTE_ACCESS - Rmap (Passive)
    - Rmap Look One (Classical LRU, MGLRU)
    - Rmap Look Around (MGLRU)
  - PTE_ACCESS - Page Table Walker (Mostly Passive) (MGLRU)
  - PTE_ACCESS - DAMON (Proactive)
  - …

Memory overcommit

- One of the best design?
- What does the kernel do when the user request more memory than physical RAM?
  - Virtual - Nah, that’s fine, not really using these mapping region.
  - Physical - on fault
    - No Swap / No Cache - > OOM
    - Kind of insane that Linux allow you to request more memory but kills you if you actually use them :)
    - With Swap